



**Università
di Genova**

DIBRIS DIPARTIMENTO
DI INFORMATICA, BIOINGEGNERIA,
ROBOTICA E INGEGNERIA DEI SISTEMI

Flatness Filtering for Honeyword Generation

Sona Hosseinzadeh Mahdavi

Master Thesis

Università di Genova, DIBRIS
Via Opera Pia, 13 16145 Genova, Italy
<https://www.dibris.unige.it/>



MSc Computer Engineering
Software and Computing Platforms Curriculum

Flatness Filtering for Honeyword Generation

Sona Hosseinzadeh Mahdavi

Advisor: Alessandro Armando

Examiner: Matteo Dell'Amico

September, 2026

Abstract

Honeyword schemes defend password databases against offline dictionary attacks by storing decoy passwords, or sweetwords, alongside the real one, so that a stolen hash file cannot be used without risking detection. Their security rests on flatness: the real password must be indistinguishable from its decoys. Dani et al. [Dan+24] have shown that a deep-learning classifier can pick the real password out of a sweetword set, breaking the flatness of existing honeyword generation techniques.

This thesis reuses that classifier defensively. A CNN flatness oracle scores each candidate sweetword set and rejects the sets in which the real password stands out, regenerating until a set passes or a regeneration cap is reached. I benchmark ten generation methods on the RockYou corpus — rule-based chaffing, HoneyGen variants, and a self-trained GAN — at training sizes of 2000, 5000, and 10000, and evaluate each against three attackers: a HoneyGen-tuned CNN, a mixed-corpus CNN, and a frequency ranker. The primary metric is the attacker’s success rate at guess budget t (ASR@ t): the fraction of accounts an attacker compromises within t guesses against the $k = 20$ sweetwords of an account, whose random-guessing baseline is t/k .

Unfiltered generators are highly vulnerable to the CNN: at $k = 20$, ASR@1 reaches 0.28–0.35, ASR@5 0.68–0.76, and ASR@10 0.86–0.91, far above the 0.05, 0.25, and 0.50 random baselines. Ratio-based filtering is the strongest defence that stays deployable, cutting these to 0.05–0.07, 0.29–0.34, and 0.58–0.63 — close to the random-guessing line at every budget — while keeping the real password near the middle of the attacker’s ranking, which requires treating the acceptance rate as a deployment requirement. The GAN is a cautionary case: its reported ASR@5 is 0.87, not because it is easy to attack under a naive CNN (there it looks near zero) but because the real password sits at the bottom of the ranking ($F_{\text{rank}} \approx 0.92$), so a rational attacker inverts the ranking and succeeds; a mixed-corpus attacker independently pushes its ASR@5 close to 1. This is inverted distinguishability, not flatness.

Keywords: Honeywords, passwords, authentication, flatness, deep learning.

Table of Contents

Chapter 1	Introduction	8
1.1	Motivation	9
1.2	Approach	9
1.3	Contributions	10
1.4	Scope and Assumptions	11
Chapter 2	Background	12
2.1	Password Guessing After a Breach	12
2.2	The Honeyword Primitive	13
2.2.1	Sweetword Storage	13
2.2.2	The HoneyChecker	13
2.2.3	Alarms and Response	14
2.3	Classical Flatness	14
2.3.1	The Attacker’s Task	14
2.3.2	$1/k$ -Flatness and ε -Flatness	15
2.3.3	Related Security Properties	15
2.4	Character-Level Convolutional Classifiers	15
2.5	FastText Password Embeddings	16
Chapter 3	Related Work	18
3.1	Heuristic Honeyword Generators	18

3.2	Representation-Learning Approaches: HoneyGen	19
3.3	Large-Language-Model Approaches	20
3.4	GAN-Based Honeyword Generators	20
3.5	Neural Distinguishers as Attackers: PassFilter	21
3.6	The HBAT Survey	22
3.7	Position of This Thesis	22
Chapter 4	Methodology	24
4.1	Overview	24
4.2	Attacker-Relative Set-Level Flatness	25
4.2.1	Sweetword Set and Attacker Model	25
4.2.2	Set-Level Induced Distribution	25
4.2.3	Flatness Metrics	25
4.2.4	Operational Definition of Flatness	27
4.3	Corpora and Splits	27
4.3.1	Sources	27
4.3.2	Disjoint Oracle and Attacker Corpora	27
4.3.3	Set-Level Splits	28
4.4	The CNN Discriminator	28
4.4.1	Architecture	28
4.4.2	Training	28
4.4.3	Dual Role and Seed Separation	29
4.4.4	Robustness Attackers	29
4.5	Honeyword Generation Methods	30
4.5.1	Chaffing Family	30
4.5.2	HoneyGen Family	31
4.5.3	GAN	31
4.6	Hybrid Flatness-Filtering Algorithm	32

4.6.1	Candidate-Level Pre-Screening	32
4.6.2	Set-Level Rejection Sampling	33
4.6.3	Algorithm	33
4.6.4	Complexity	34
4.7	Evaluation Protocol	35
4.7.1	Design	35
4.7.2	Rational Attacker Floor	35
4.8	Experimental Protocol and Parameters	36
4.8.1	Locked Parameters	36
4.8.2	Threshold Selection and the Accept-Rate / ASR Trade-off	36
4.8.3	Metrics and Statistical Treatment	37
4.8.4	Robustness Check	37
Chapter 5	Results	38
5.1	Setup	38
5.2	Unfiltered Baselines Under the CNN Attacker	39
5.3	Effect of Flatness Filtering	40
5.4	Cost of Filtering and the Deployability Gate	41
5.5	Stability Across Training-Set Size	43
5.6	ASR Curves and Consistency Across Test-Set Size	43
5.7	The GAN Result Requires a Caveat	43
5.8	Robustness Across Attacker Class	46
5.9	Discussion	47
Chapter 6	Conclusion	49
6.1	Summary of Contributions	50
6.2	What the Numbers Actually Say	50
6.3	Limitations	51

6.4	Open Questions	52
6.5	Closing	53

Chapter 1

Introduction

Password authentication remains the dominant credential mechanism for consumer web services, in spite of decades of proposals for alternatives. Its persistence is a consequence of deployability, familiarity, and a near-zero marginal cost. Its cost is a long history of database breaches that repeatedly expose hundreds of millions of hashed credentials to offline guessing attacks. Once a hash file has left the server, the guessing rate is bounded only by the attacker’s compute, and any weakness in the underlying password distribution translates directly into recovered plaintexts.

Juels and Rivest [JR13] proposed *honeywords* as a detection layer over this failure mode. For each account the authentication server stores a *sweetword set* of k candidate hashes, one of which is the real password and the other $k - 1$ of which are decoys, together with a separate index held by a hardened *HoneyChecker* that identifies which position is real. A successful login with a decoy is a signal that the hash file has been read offline and the attacker has begun trying candidates. The scheme does not prevent guessing; it converts silent database compromise into an event that the server can detect and act on.

The security of the scheme rests on a property called *flatness*: given the sweetword set, an attacker who has read the hash file must not be able to identify the real password with probability much better than $1/k$. Under perfect flatness, all k candidates are indistinguishable to the attacker, and every login attempt against a decoy raises the alarm with the same probability. Flatness is what turns honeywords from a labeling exercise into a detection guarantee.

1.1 Motivation

Flatness, in most of the honeyword literature, is measured against attackers that no longer represent the state of the art. Traditional evaluations rank sweetwords using frequency counts, probabilistic context-free grammars, or Markov models, and honeywords are declared flat when the real password does not stand out under those rankers. Dani et al. [Dan+24] showed that a modestly sized character-level convolutional neural network trained as a binary classifier defeats this notion on the honeyword schemes it was tested against, including the representation-learning generators of Dionysiou, Vassiliades, and Athanasopoulos [DVA21]. Under a learned discriminator, the attacker success rate is substantially higher than $1/k$, sometimes by an order of magnitude, and sweetwords that pass heuristic evaluation turn out to be individually plausible but collectively distinguishable.

This gap has two consequences that motivate the rest of the thesis. First, the current honeyword generators have almost never been benchmarked against a neural attacker, so it is not known how badly they fail once one is admitted into the threat model. Second, no construction in the literature restores flatness against such an attacker without redesigning the generator itself, which would leave existing deployments unprotected. Both issues need to be resolved for honeyword-based breach detection to remain a credible defense.

1.2 Approach

We take the same discriminator that Dani et al. [Dan+24] introduced as an attack tool and reuse it, unmodified in architecture, as a *defensive oracle* during sweetword construction. A candidate sweetword set is produced by any of the standard mechanisms, scored by the oracle, and either accepted or discarded depending on whether the induced score distribution over the set satisfies a small collection of measurable flatness conditions. Sets that fail are regenerated up to a bounded budget. The classifier that would otherwise identify the real password thus becomes the gatekeeper that prevents any set with an identifiable real password from being stored.

The construction is deliberately generator-agnostic. We apply it as a post-processing layer over ten honeyword generation methods, spanning rule-based tweaking, representation-learning approaches derived from Dionysiou, Vassiliades, and Athanasopoulos [DVA21], and a self-trained GAN. Each method is trained once at a fixed training-set size and evaluated on a common test set of sweetword sets, so that comparisons across generators are made on the same footing. The primary evaluation uses a CNN attacker trained on HoneyGen; robustness is then checked against two further attacker classes – a mixed-corpus CNN trained on several decoy families and a frequency ranker – which together distinguish genuine flatness from single-attacker artifacts.

The primary security metric is the attacker’s success rate at guess budget five, ASR@5: the fraction of accounts an attacker compromises when it is allowed five guesses among the $k = 20$ sweetwords of an account. We report the full curve ASR@ t for $t = 1, \dots, 20$, but summarise on $t = 5$ because it matches the honeyword threat model. After cracking a stolen hash file offline, an attacker still has to submit a recovered sweetword to the live service to use it; submitting a honeyword trips the honeychecker, and most online services additionally lock an account after a small number of failed attempts, with five a common threshold [Dan+24]. ASR@5 therefore measures what an attacker can achieve within one realistic lockout window. The extremes are less informative as a single headline: ASR@1 reflects only the single best guess, while ASR@20 equals 1.0 by construction once all $k = 20$ sweetwords have been tried. For a perfectly flat set the attacker does no better than random, giving $\text{ASR@5} = 5/20 = 0.25$.

1.3 Contributions

The thesis makes the following contributions.

1. **Attacker-relative set-level flatness.** We define flatness as a property of the probability distribution that a learned attacker induces over a sweetword set, and we characterize it by four quantities computed from that distribution: rank-based flatness, normalized entropy, top and real gaps, and an ε -flatness bound on the largest renormalized score in the set. These replace the frequency-based flatness notions that dominate the earlier literature and make the property operational under any binary discriminator.
2. **Hybrid flatness filtering.** We propose a two-stage rejection-sampling algorithm that combines candidate-level pre-screening with set-level acceptance. Pre-screening removes individually distinguishable decoys before a set is assembled; set-level acceptance discards sets whose induced distribution violates any of the flatness conditions. The algorithm operates as a post-processing layer and does not require modifying the underlying generator or retraining the attacker.
3. **Benchmark of ten honeyword generators against a neural attacker.** We evaluate Chaffing-by-Tweaking, Chaffing-with-Password-Model, Chaffing-with-Hybrid-Model, HoneyGen Baseline, HoneyGen Set-Filter, HoneyGen Prescreen-Add, HoneyGen Prescreen-Ratio, HoneyGen Hybrid-Add, HoneyGen Hybrid-Ratio, and a self-trained GAN, on the RockYou corpus with $k = 20$. Reporting the same attacker success rate under a common experimental protocol lets us separate genuine flatness gains from distributional artifacts of the generator, an effect that turns out to matter for the GAN and that we discuss in Chapter 5.

4. **A defensive reinterpretation of the neural-attacker paradigm.** The thesis shows that a discriminator of the type used by Dani et al. [Dan+24] as an attack can be repurposed, without architectural change, into the enforcement mechanism of a flatness guarantee. What defeats current honeyword schemes at test time becomes what protects them at storage time.

1.4 Scope and Assumptions

The guarantees this thesis provides are model-relative. Filtering is performed against one specific attacker, the character-level CNN described in Chapter 4, and the flatness conditions constrain that attacker’s induced distribution over the sweetword set. A different attacker, with a different architecture or with access to side information beyond the sweetword strings, may recover distinguishability. Chapter 6 returns to this point when discussing the limits of the framework.

The evaluation is restricted to leaked passwords from the RockYou corpus and to sweetword sets of size $k = 20$. Only the password string itself is used as input by the generators and the attackers. Chapter 5 reports how far the conclusions carry over across attacker classes, from the HoneyGen-tuned CNN to a mixed-corpus CNN and a frequency ranker; extending the benchmark to a second corpus such as *darkc0de* is left as future work (Chapter 6).

The remainder of the thesis proceeds as follows. Chapter 2 sets out the honeyword primitive, the HoneyChecker architecture, and the character-level convolutional networks that both the attack and the defense rely on. Chapter 3 reviews the honeyword generators we benchmark against and situates Dani et al. [Dan+24] within the wider line of neural password modeling. Chapter 4 develops the flatness formalism, the CNN training procedure, the ten generators, the two-stage filtering algorithm, and the evaluation protocol. Chapter 5 reports the ASR curves across the ten generators, isolates the effect of filtering on both security and the flatness criteria, weighs it against the acceptance rate as a deployability requirement, checks robustness across attacker classes, and discusses a distributional artifact in the GAN results that limits how far its low ASR should be trusted. Chapter 6 draws the argument together and lists the open questions the framework leaves behind.

Chapter 2

Background

This chapter collects the definitions and mechanisms that the rest of the thesis takes as given. Section 2.1 describes how offline password guessing works and what a breach exposes. Section 2.2 states the honeyword primitive introduced by Juels and Rivest [JR13] and the HoneyChecker architecture that supports it. Section 2.3 formalizes the classical notion of flatness relative to a fixed attacker class. Section 2.4 introduces the character-level convolutional networks that Chapter 3 and Chapter 4 both build on, and Section 2.5 covers the FastText subword embeddings used by the HoneyGen family of generators.

Every concept is presented at the level required to follow the argument in later chapters. The formal set-level flatness definition that this thesis introduces is deferred to Section 4.2, since it is part of the contribution rather than of the background.

2.1 Password Guessing After a Breach

Password authentication systems store a transformation of the user’s credential rather than the credential itself. In modern deployments this transformation is a salted, slow cryptographic hash such as bcrypt, scrypt, or Argon2. When a user submits a password w at login, the server computes the hash and compares it to the stored value; a match authenticates the session.

Two attack models are relevant here. In the *online* model the attacker submits guesses through the authentication interface, and the server can rate-limit or lock accounts after a small number of failed attempts. In the *offline* model the attacker has obtained a copy of the password file and can test guesses locally at the rate their hardware allows. Modern GPU rigs push this rate high enough that the security of the deployment is determined almost entirely by the password’s position in the guessing order used by the attacker.

The guessing order itself has been the subject of a long research line. Narayanan and Shmatikov [NS05] showed that Markov models of character sequences accelerate cracking substantially over dictionary attacks. Weir et al. [Wei+09] extracted probabilistic context-free grammars from leaked corpora and produced guesses in decreasing order of estimated probability. Melicher et al. [Mel+16] used neural language models over character sequences to reach guessing rates that outperform both Markov and PCFG models on realistic budgets. All three lines share the same interface: given a candidate password, produce a score or a rank that reflects how likely it is to appear in a real distribution.

The consequence for defenders is that a database compromise is silent. Nothing distinguishes an offline guessing campaign from a period in which no attacker has the hash file at all, until the attacker’s recovered credentials are used against a live login. Honeywords, the subject of the next section, are designed to break that silence.

2.2 The Honeyword Primitive

2.2.1 Sweetword Storage

Juels and Rivest [JR13] propose the following modification to the password file. For each user account the server stores not a single hash but a set of k hashes, one of which corresponds to the real password and the other $k - 1$ to *honeywords*: decoy strings generated to be difficult to distinguish from the real password. We call the set of k candidates the *sweetword set*, and any element of it a *sweetword*. The honeywords are not stored in clear anywhere; only their hashes appear in the password file.

The order of the sweetwords in the file is either randomized per user or made independent of the identity of the real password. The information about which position corresponds to the real password is stored elsewhere, in a component we describe next.

2.2.2 The HoneyChecker

The scheme separates two pieces of information that were previously combined in a single password file: the credential material (the k hashes) and the identity of the correct one (an index in $\{1, \dots, k\}$). The credential material stays on the main authentication server, and the index is held by a second, hardened component called the *HoneyChecker*. The HoneyChecker exposes a narrow interface: given a user identifier and an index, it returns whether the index is the correct one, and it raises an alarm if the index is not.

An honest login proceeds as follows. The user submits their password. The authentication server hashes it and looks for a match among the k stored sweetword hashes. If a match is

found at index j , the server queries the HoneyChecker with that index. The HoneyChecker returns *accept* when j is the correct index, and the login succeeds. This is indistinguishable to the user from an ordinary password check.

A honeyword-triggered login proceeds differently. The attacker, who has read the password file offline and recovered one of the k hashes, attempts a login using the recovered plaintext. The authentication server finds a match at some index j' and queries the HoneyChecker. If j' is not the correct index, the HoneyChecker returns *reject* and, in addition, raises an alarm. The alarm is what makes the primitive a breach detector: a decoy login is observable in real time, whereas a successful compromise otherwise is not.

The distributed trust assumption is that a single adversary does not compromise both the password file and the HoneyChecker. Reading only the password file leaves the attacker with k indistinguishable hashes; compromising only the HoneyChecker gives away indices with no credential material to attach them to. This assumption is what makes the scheme meaningful; Juels and Rivest [JR13] and the subsequent survey by Chakraborty et al. [Cha+22] discuss its practical implications at length, and we take it as given throughout the thesis.

2.2.3 Alarms and Response

Once the HoneyChecker raises an alarm, the response is a policy decision outside the scope of this thesis. Depending on the deployment, the response can lock the affected account, force a password reset for a class of users, or trigger a wider incident investigation. The relevant property here is that the alarm is a signal that the server did not have before the primitive was in place.

2.3 Classical Flatness

2.3.1 The Attacker’s Task

Given only the sweetword set $S = \{w_1, \dots, w_k\}$, the attacker must decide which element is the real password. This is a k -way distinguishing problem. If the honeywords are well constructed, the attacker’s success probability should be close to $1/k$, the rate of uniform random guessing; if the honeywords are distinguishable, the success probability is higher.

The exact success probability depends on the attacker’s knowledge and inference capabilities. Juels and Rivest’s original paper models the attacker as a probabilistic ranker: a function A that, given a candidate password, returns a probability that it comes from the

distribution of real user-chosen passwords. The attacker orders the k sweetwords by A 's output and guesses the top-ranked one.

2.3.2 $1/k$ -Flatness and ε -Flatness

Let A be an attacker and let $p_A(S)$ denote the probability that A identifies w^* on its first guess, taken over the random choice of S and the internal randomness of the generator. The scheme is called $1/k$ -flat against A if

$$p_A(S) = 1/k, \quad (2.1)$$

which is the uniform-random rate. Since perfect equality is rarely attainable, the literature relaxes the condition to ε -flatness: the scheme is ε -flat if

$$p_A(S) \leq \frac{1 + \varepsilon}{k}, \quad (2.2)$$

for some fixed $\varepsilon > 0$. The two definitions coincide as $\varepsilon \rightarrow 0$.

Both definitions are attacker-relative in principle: they depend on the class of function A used. The classical literature restricts A to the probabilistic ranker family described above. Chapter 4 extends the definition to arbitrary binary discriminators and formalizes it at the level of the induced distribution over the set; that extension is one of the contributions of this thesis.

2.3.3 Related Security Properties

Chakraborty et al. [Cha+22] organize the honeyword security literature around three properties: flatness, denial-of-service (DoS) resistance, and multiple-system vulnerability (MSV) resistance. DoS resistance controls the rate at which a legitimate but confused user can be locked out by accidentally submitting a honeyword. MSV resistance controls the extent to which learning the sweetwords for one user helps an attacker for other users. This thesis addresses only flatness. The DoS and MSV properties are largely orthogonal to the filtering construction we develop, and we return to them briefly in Chapter 6.

2.4 Character-Level Convolutional Classifiers

Character-level convolutional networks are the standard architecture for both attack and defense in this thesis. The design is a small adaptation of the text classifier introduced by Kim [Kim14] and the character-level variants of Zhang, Zhao, and LeCun [ZZL15].

A password w of at most L characters is right-padded to length L and each character is mapped to an embedding vector of dimension d through a lookup table. The resulting $L \times d$ matrix is treated as a one-dimensional signal with d input channels. One or more one-dimensional convolutional layers extract features from local n -character windows: each convolution with kernel width m applies a learned filter to every window of m consecutive positions and produces a new feature map. A ReLU activation follows each convolution. A max-pooling step reduces the length dimension, and a fully connected layer projects the pooled features to the number of output classes. A final softmax converts the logits to class probabilities.

For the binary classifier used here, the two output classes are *real* and *honeyword*, and the quantity of interest is $P(\text{real} \mid w)$, the probability the network assigns to the real class for the input password. This score is the raw material for every flatness measurement in the thesis. The precise architecture and training procedure are specified in Section 4.4.

The property that makes this class of network well suited to password strings is that its filters operate on local character windows. This matches the empirical observation that real user-chosen passwords concentrate distinguishing structure at the level of short substrings (digit sequences, common suffixes, case patterns), which the convolutional layers pick up efficiently without requiring token-level segmentation.

2.5 FastText Password Embeddings

The HoneyGen generator family described in Section 4.5 and reviewed in Chapter 3 uses FastText embeddings as its representation of the password space. FastText, introduced by Bojanowski et al. [Boj+17], extends the skipgram word representation of Mikolov et al. [Mik+13] by representing each token as the sum of embeddings for its character n -grams. A password like `qwerty2020` is represented not only by an embedding for the string as a whole but by the sum of embeddings for all its character n -grams of length in a configured range.

Two properties of this representation matter for honeyword generation. First, the model produces embeddings for out-of-vocabulary strings, which is essential because a generator is asked to produce passwords it has never seen. Second, the nearest neighbors of a password in embedding space tend to share sub-word structure: a password ending in `2020` is close to passwords ending in `2021`, `2019`, and so on. This makes the embedding a source of plausible perturbations of a real password without requiring hand-written rules.

The HoneyGen family uses a skipgram FastText model trained on the oracle corpus, with the smallest possible `minCount` and a minimum n -gram length of two. Section 4.5 gives the exact configuration used in the experiments; the description here is the background

needed to follow that section.

Chapter 3

Related Work

The literature on honeyword-based authentication has two threads. The first is the design of *generators* that produce honeywords intended to be difficult to distinguish from real passwords. The second, more recent, is the design of *distinguishers* that attempt to break flatness under stronger attacker models than the ones generators are usually evaluated against. Both threads inform this thesis: the ten generators of Section 4.5 are drawn from the first thread, and the CNN of Section 4.4 is drawn from the second. This chapter reviews both, and Section 3.7 describes how the filtering construction of Section 4.6 sits between them.

3.1 Heuristic Honeyword Generators

The earliest honeyword generators are the four *chaffing* constructions proposed alongside the primitive by Juels and Rivest [JR13]: *chaffing-by-tweaking*, *chaffing-by-tail-tweaking*, *chaffing-with-a-password-model*, and *chaffing-with-tough-nuts*. Chaffing-by-tweaking applies character-class substitutions to the real password to produce visually similar decoys; chaffing-by-tail-tweaking restricts the edits to a short suffix. Chaffing-with-a-password-model draws candidates from an external password model such as a PCFG or a Markov chain, so that the decoys look like plausible passwords in the aggregate rather than like edits of the real one. Chaffing-with-tough-nuts inserts random, high-entropy strings that no attacker would submit at login and that therefore serve as guaranteed alarms.

These constructions were evaluated against attackers that scored candidates independently, typically PCFG-based rankers of the kind introduced by Weir et al. [Wei+09] and Markov rankers of the kind introduced by Narayanan and Shmatikov [NS05]. The flatness numbers reported under this evaluation model are close to $1/k$ for carefully-tuned chaffing schemes,

which motivated the follow-up literature. Two of these constructions, chaffing-by-tweaking and chaffing-with-a-password-model, appear in our benchmark (Section 4.5) as reference points against the more sophisticated generators discussed below, together with a hybrid chaffing method that mixes the two and that we introduce for comparison.

The main line of criticism against the heuristic generators is *per-password plausibility*: the decoys are designed so that each one looks like a plausible password in isolation, but there is no constraint on how they look *as a set*. A generator can produce $k - 1$ plausible but distributionally-clustered decoys that leave the real password stranded, and the attacker only needs to notice the cluster to identify the outlier. This is the phenomenon that later generators, and the filter of Section 4.6, are designed to address.

3.2 Representation-Learning Approaches: HoneyGen

Dionysiou, Vassiliades, and Athanasopoulos [DVA21] propose *HoneyGen*, a generator that combines a FastText password model with rule-based tweaking. The password model produces nearest-neighbor candidates for the real password in the FastText embedding space (Section 2.5), and a tweaking module produces additional candidates by character-class substitution. The two streams are merged, pruned using a frequency-based filter that removes overly common candidates, and the top $k - 1$ are retained as honeywords.

The evaluation in the original paper is against frequency-based and PCFG-based attackers, and HoneyGen achieves flatness scores that improve on the chaffing baselines. Under this class of attacker the paper’s conclusion is uncontroversial: representation learning captures more of the password distribution than either rules or PCFGs alone, and the resulting decoys are harder to distinguish under a ranker of the kind those attackers rely on.

Our position with respect to HoneyGen is precise. We reuse the generator, without modification, as the source of candidates for six of the ten methods in the benchmark (HoneyGen Baseline, Set-Filter, Prescreen-Add, Prescreen-Ratio, Hybrid-Add, Hybrid-Ratio). What we do not reuse is the paper’s evaluation: our attacker is the CNN of Section 2.4 rather than a frequency ranker, and the flatness scores we report under that attacker are substantially worse than the ones the original paper reports (Chapter 5). The four filtered variants are the mechanism by which we recover flatness against this stronger attacker without modifying the generator itself.

3.3 Large-Language-Model Approaches

More recent work uses large language models as the source of honeyword candidates. The chunk-level construction of Yu and Martin [YM23] has drawn particular attention: a password is disassembled into semantic chunks that a pre-trained language model perturbs one at a time, and candidate sets are drawn with Metropolis–Hastings sampling – an accept-or-reject sampling scheme that steers the generated candidates toward regions the model assigns a high real-password probability – while locality-sensitive hashing, a technique that maps similar strings to the same bucket, is used to drop near-duplicates from the pool. These are among the most linguistically sophisticated generators the honeyword literature has produced, and they report clear gains in semantic plausibility over both HoneyGen and GAN-based methods.

The evaluation in this line, however, follows the same statistical plausibility paradigm as the earlier work. Flatness is measured against ranker-style attackers, and the metrics reported are distributional distances rather than the success rate of a neural distinguisher trained specifically to identify the real password. The gap this leaves is exactly the one Dani et al. [Dan+24] demonstrated for HoneyGen: a generator can be indistinguishable under one attacker class and yet break under a stronger one. We do not evaluate an LLM-based generator directly in this thesis, since our focus is on the reuse of the neural distinguisher as a defensive oracle rather than on the generation side. The construction of Section 4.6 is agnostic to the source of candidates and would apply to an LLM-based generator without modification; Chapter 6 returns to this point.

3.4 GAN-Based Honeyword Generators

Hitaj et al. [Hit+19] adapt the generative adversarial network of Goodfellow et al. [Goo+14] to password modeling, in the attack setting: the goal is to produce guess sequences that recover more of a leaked corpus than PCFG or Markov models produce at the same budget. The adaptation is to train the generator against a discriminator that tries to separate generated strings from real leaked ones, with both networks operating on character sequences.

GANs have already been studied as a source of honeyword candidates. Fauzi, Yang, and Martiri [FYM20] analyse a game in which the defender uses the generator of a PassGAN model to produce honeywords while the attacker uses its discriminator to pick out the real password, and assess the feasibility of PassGAN as a honeyword generation strategy. We include a GAN in this same spirit as one of the generators in our benchmark: the intuition is that a model that has learned the password distribution should, in principle, produce decoys that look like real passwords. Our GAN is a character-level generator trained from scratch on the same corpus as the other generators (a *self-trained* GAN), and we sample

decoys from it directly. GANs are prone to *mode collapse*, where the generator learns to emit only a narrow set of outputs instead of covering the full variety of the training distribution; for passwords this shows up as the same few common strings being produced over and over. To obtain enough distinct decoys despite this, we oversample the generator – drawing roughly five times as many candidates as a set needs – and deduplicate the resulting pool before assembling each sweetword set. This GAN is one of the ten generators evaluated in our benchmark (Chapter 4).

The theoretical case for using a GAN as a honeyword generator is subtle, and our results show it does not hold in practice. A generator that had truly converged to the real password distribution would produce indistinguishable decoys almost by definition, but real password distributions have long low-probability tails that GANs are known to underrepresent, so the resulting decoys cluster in the high-probability region and the real password, often itself a low-probability string, is left looking anomalous. In Chapter 5 we show exactly this: under the CNN attacker the GAN’s real password is pushed to the bottom of the ranking ($F_{\text{rank}} \approx 0.92$), so its apparently low attack success is an artifact of that inversion rather than genuine flatness, and a mixed-corpus attacker exposes it. This is why the GAN’s flatness numbers must be read with care rather than taken at face value.

3.5 Neural Distinguishers as Attackers: PassFilter

The line of work most directly relevant to this thesis is PassFilter [Dan+24]. It trains a character-level convolutional neural network as a binary classifier that predicts, given a candidate password, whether it is real or a honeyword. The network is small by modern standards (two convolutional layers over character embeddings, followed by a fully connected layer), and it is trained on labeled pairs of real passwords and honeywords produced by existing generators.

The main result of the paper is that this classifier violates flatness on every honeyword scheme they test, including HoneyGen and its variants, on multiple leaked corpora. The attacker success rate is substantially higher than $1/k$ across the board, sometimes by an order of magnitude, and the ranking induced by the classifier concentrates on the real password with a probability the classical flatness definition explicitly forbids.

Using a learned discriminator as the attacker is not unique to PassFilter. As noted in Section 3.4, Fauzi, Yang, and Martiri [FYM20] already turn the discriminator of a PassGAN model against its own generator’s honeywords; PassFilter can be read as the general form of the same idea, a classifier trained to separate real passwords from honeywords regardless of which generator produced them. Both establish the neural distinguisher as the attacker class that honeyword schemes must withstand. This thesis takes that distinguisher and moves it to the defensive side.

Our reading of the PassFilter result is that the honeyword literature has been evaluating flatness against too weak an attacker class. The constructive response we develop is to admit the same class of classifier into the threat model, formalize flatness against it (Section 4.2), and then reuse the classifier itself, without architectural change, as the enforcement mechanism during sweetword construction (Section 4.6). The name we give to this reuse, *defensive oracle*, is meant to underline that the same object is used offensively in PassFilter and defensively in this thesis; only the surrounding pipeline changes.

Two aspects of the PassFilter setup differ from ours in ways that matter. First, the paper releases model weights that are trained on a specific corpus and are used to attack sweetwords produced by generators trained on the same corpus. We do not reuse those weights; we train our own CNN on a disjoint attacker corpus (Section 4.3), which raises the baseline attacker success rate but removes any shared-parameter bias between attacker and generator. Second, the paper evaluates flatness at the level of individual sets. We report the full ASR curve $\text{ASR}(t)$ over the guess budget t , computed over the accepted sets, for every generator under a common attacker.

3.6 The HBAT Survey

Chakraborty et al. [Cha+22] survey the honeyword literature and propose a framework, *honeyword-based authentication techniques* (HBAT), that organizes the existing schemes along three axes: generation method, distributed security model, and evaluation metric. Their treatment of flatness is compatible with, and indeed formalizes, the definition given by Juels and Rivest [JR13]; they extend it to ε -flatness and connect it to two further security notions (DoS resistance and MSV resistance) that we mentioned in Section 2.3.

The survey’s attacker taxonomy runs from frequency-based rankers through PCFG and Markov models to neural language models. Neural *distinguishers* of the kind introduced by Dani et al. [Dan+24] appear implicitly in the survey’s discussion of adversarial machine learning but are not integrated into the flatness definition itself. The refinement of the flatness definition in Section 4.2 can be read as such an integration.

3.7 Position of This Thesis

The construction developed in this thesis sits at the intersection of the two threads reviewed above. From the generator thread, it inherits the ten methods of Section 4.5 without modification. From the distinguisher thread, it inherits the CNN classifier of Dani et al. [Dan+24], again without architectural change. What is new is the pipeline in which the two are combined: the classifier is trained on a disjoint corpus and reused as the oracle that

decides which sweetword sets are stored, and the flatness conditions the oracle enforces are stated at the level of the set-wide induced distribution defined in Section 4.2 rather than at the level of individual candidates.

Three summary observations situate the contribution.

1. Prior generators are evaluated against attackers that they themselves inspired. Frequency-based generators are evaluated against frequency rankers, and representation-learning generators are evaluated against Markov or PCFG rankers. This thesis holds the attacker class fixed, at the CNN of Dani et al. [Dan+24], and lets the generators vary; that common attacker is what makes the ten methods comparable on a single ASR axis. Robustness to the choice of attacker is then examined separately (Section 5.8), by re-evaluating the accepted sets against a mixed-corpus CNN and a frequency ranker.
2. Prior work on neural distinguishers stops at the demonstration that flatness is violated. It does not propose a construction that restores flatness. This thesis proposes one and measures how far it goes.
3. Prior filtering ideas exist in the honeyword literature, but they operate at the candidate level and use non-learned filters (frequency pruning, dictionary matching). The two-stage filter of Section 4.6 uses a learned filter and, in its set-level stage, constrains the distribution rather than the individual candidates. The combination is what allows the filter to enforce ε -flatness in the sense of Equation (4.7).

The remainder of the thesis develops these three points in detail. Chapter 4 specifies the construction, and Chapter 5 measures its effect on ASR across the ten generators.

Chapter 4

Methodology

This chapter describes the construction we use to enforce set-level flatness against a learned attacker. Section 4.1 states the pipeline and its main design choices. Section 4.2 formalizes the attacker-induced distribution and the flatness metrics computed on top of it. Section 4.3 describes the corpora and the disjoint oracle/attacker split. Section 4.4 specifies the character-level CNN that plays the dual role of defensive oracle during filtering and adversary during evaluation. Section 4.5 describes the ten honeyword generators we benchmark. Section 4.6 presents the hybrid flatness-filtering algorithm. Section 4.7 describes the evaluation protocol and the rational-attacker floor applied to reported ASR. Section 4.8 lists the locked parameters used by every experiment in Chapter 5.

4.1 Overview

For each real password w^* , a generator produces $k - 1$ candidate honeywords which are combined with w^* into a sweetword set $S = \{w_1, \dots, w_k\}$ of size $k = 20$. Two independently trained character-level CNNs interact with S at different points in the pipeline. The first, the *oracle*, is used at storage time by the filtering algorithm to decide whether S satisfies the flatness conditions defined in Section 4.2 and, if not, to trigger regeneration. The second, the *attacker*, is used at evaluation time to measure the attacker success rate on the accepted sets. The two networks share the same architecture but are trained on disjoint corpora with different random seeds (Section 4.4), which is what allows us to report ASR against an attacker that is not the model the filter was tuned against.

Filtering is performed once, at password storage time, and is never invoked during authentication. The construction is generator-agnostic: we apply the same filter, with the same thresholds, on top of ten distinct honeyword generators, including one (HoneyGen

Baseline) that performs no filtering and serves as the reference point for measuring the effect of the filter.

4.2 Attacker-Relative Set-Level Flatness

4.2.1 Sweetword Set and Attacker Model

A sweetword set is a size- k multiset $S = \{w_1, \dots, w_k\}$ in which exactly one element w^* is the real password and the remaining $k - 1$ elements are honeywords produced by a generator. The attacker A is a binary classifier trained on labeled examples of the form (w, y) with $y = 1$ for real passwords and $y = 0$ for honeywords. For every candidate w_i , the classifier outputs an independent score

$$s_i = s(w_i) = P_A(y = 1 \mid w_i) \in [0, 1]. \quad (4.1)$$

The scores are per-candidate probabilities from a binary classifier, not a distribution over S . In particular $\sum_i s_i \neq 1$ in general, and the classifier processes each w_i without seeing any other member of S .

4.2.2 Set-Level Induced Distribution

The attacker’s actual decision problem is comparative: given all of S , choose one candidate. We model this comparison by renormalizing the per-candidate scores to form a distribution over S :

$$q_i = \frac{s_i}{\sum_{j=1}^k s_j}, \quad \sum_{i=1}^k q_i = 1. \quad (4.2)$$

The vector $q = (q_1, \dots, q_k)$ is the *attacker-induced distribution* on S . Under ideal flatness q is uniform, so $q_i = 1/k$ for every i . We write q_{real} for the entry of q corresponding to w^* and use it as the central quantity that the filter constrains.

4.2.3 Flatness Metrics

Four quantities computed from the pair (s, q) characterize how far S deviates from flatness under A .

Rank-based flatness. Order the candidates by s_i in decreasing order and let $\text{rank}(w^*)$ be the position of w^* in that order. We define

$$F_{\text{rank}} = \frac{\text{rank}(w^*) - 1}{k - 1} \in [0, 1]. \quad (4.3)$$

A value of 0 means the attacker ranks w^* first (worst case for the defender); a value of 1 means it ranks w^* last; the expected value under a uniform ranker is $1/2$.

Normalized entropy. The Shannon entropy of q divided by its maximum $\log k$:

$$F_{\text{ent}} = \frac{H(q)}{\log k} = -\frac{1}{\log k} \sum_{i=1}^k q_i \log q_i. \quad (4.4)$$

$F_{\text{ent}} = 1$ if and only if q is uniform. Entropy captures global smoothness of q but does not by itself constrain which entry of q is the largest, and a set can have F_{ent} close to 1 while still assigning the top rank to w^* . We therefore treat F_{ent} as a secondary stabilizing condition rather than a primary one.

Gap metrics. Let $s_{(1)} \geq s_{(2)} \geq \dots \geq s_{(k)}$ be the sorted raw scores. The top gap

$$\Delta_{\text{top}} = s_{(1)} - s_{(2)} \quad (4.5)$$

measures how strongly the attacker prefers its top candidate over the runner-up, and the real gap

$$\Delta_{\text{real}} = s_{(1)} - s(w^*) \quad (4.6)$$

measures how far the real password falls below the top score. Gap metrics are computed on raw scores rather than on q because ranking is invariant under the positive rescaling of Equation (4.2) and the raw scores preserve the absolute magnitude of the classifier's preference.

ε -flatness. Let $q_{\text{max}} = \max_i q_i$ denote the largest entry of the induced distribution. The set S is ε -flat under A if

$$q_{\text{max}} \leq \frac{1}{k} + \varepsilon. \quad (4.7)$$

Setting $\varepsilon = 0$ forces $q_{\text{max}} \leq 1/k$, which together with $\sum_i q_i = 1$ recovers exact uniformity of the whole distribution; increasing ε relaxes the bound. Because $q_{\text{real}} \leq q_{\text{max}}$, this constraint upper-bounds the attacker's posterior on every sweetword, including the true credential, and is the primary security constraint enforced by the filter. This additive slack ε on the maximum entry differs from the classical multiplicative bound $(1 + \varepsilon)/k$ of Section 2.3: the

two coincide at $\varepsilon = 0$, but for $\varepsilon > 0$ the additive form places the threshold at $1/k + \varepsilon$ rather than $(1 + \varepsilon)/k$, so the same numerical ε is a looser bound here. We adopt the additive form because it acts directly on the peak of the attacker-induced distribution over the set, whereas the classical bound is stated on the attacker’s first-guess success probability.

Attacker success rate. Given the ranking induced by s , the attacker success rate at rank t is

$$\text{ASR@}t = \Pr[\text{rank}(w^*) \leq t], \quad (4.8)$$

where the probability is taken over the sample of evaluation sets. Under ideal flatness $\text{ASR@}t = t/k$. We report $\text{ASR@}1$, $\text{ASR@}3$, and $\text{ASR@}5$ throughout Chapter 5, with $\text{ASR@}5$ as the primary security metric.

4.2.4 Operational Definition of Flatness

The set S is declared flat under the attacker A if it satisfies Equation (4.7) together with two auxiliary conditions on F_{ent} and Δ_{top} . The three thresholds and their justification are given in Section 4.8. Sets that fail any of the three conditions are rejected by the filter of Section 4.6.

4.3 Corpora and Splits

4.3.1 Sources

The corpus is the cleaned RockYou leak, from which we draw disjoint oracle and attacker subsets after deduplication, encoding validation, and truncation to the CNN’s input length. The oracle corpora are the three training sizes $N_{\text{train}} \in \{2000, 5000, 10000\}$ used throughout Chapter 5; the attacker corpus is a fixed disjoint subset of 5000 passwords, split 80/10/10 and reused unchanged for every oracle size (the training composition is detailed in Section 4.4). All entries are unprocessed leaked passwords; no case folding, character substitution, or semantic normalization is applied beyond the length truncation. Extending the benchmark to a second corpus such as *dark0de* is left as future work (Chapter 6).

4.3.2 Disjoint Oracle and Attacker Corpora

The pipeline uses two disjoint corpora. The *oracle corpus* supplies the real passwords w^* from which sweetword sets are built, and also provides the training data for the oracle

CNN used by the filter; a separate oracle is trained for each generator size N_{train} . The *attacker corpus*, drawn from the same source but disjoint from the oracle corpus, provides the training data for the attacker CNN used to measure ASR on the accepted sets. Unlike the oracle, the attacker is trained once on the training split of a fixed 5000-password attacker corpus (Section 4.4) and reused across every generator size, so that a single, consistent adversary evaluates all of them. Because the attacker is trained on passwords that neither the generators nor the oracle have seen, its evaluation performance reflects generalization to unseen inputs rather than memorization of the sets it will later be asked to rank. This choice raises the baseline ASR compared to a shared-corpus setup, but the resulting numbers are the ones that matter for a real deployment.

4.3.3 Set-Level Splits

Within the oracle corpus, real passwords are partitioned into training, validation, and test subsets. Sweetword sets are constructed for every real password in each subset, and all sets are concatenated into a single evaluation stream with globally unique set identifiers. This `EVAL_FULL = True` configuration is required to avoid staircase artifacts in the ASR curve at small evaluation sizes, which would otherwise arise from evaluating only the tail of a single split.

4.4 The CNN Discriminator

4.4.1 Architecture

The classifier used for both oracle and attacker roles is a character-level convolutional network. Each password is truncated or right-padded to a fixed length and mapped through a character embedding of dimension 64. Two one-dimensional convolutional layers with ReLU activations extract local n -gram features, followed by max-pooling to reduce dimensionality. A fully connected layer projects to a two-dimensional output over which a softmax produces $P(\text{real} \mid w)$ and $P(\text{honey} \mid w)$. The architecture is inspired by the learned-attacker paradigm of Dani et al. [Dan+24] but is trained independently and does not reuse any released weights.

4.4.2 Training

The classifier is trained as a binary classifier with cross-entropy loss

$$\mathcal{L} = -(y \log s + (1 - y) \log(1 - s)). \quad (4.9)$$

Positive examples are real passwords and negative examples are honeywords produced by the HoneyGen Baseline generator described in Section 4.5.2, which combines the FastText password model with rule-based tweaking. Training the classifier against HoneyGen negatives calibrates it to the dominant family of decoys studied in this thesis and produces a strong and realistic adversary. The oracle is trained on the training split of its own N_{train} corpus. The attacker is trained once on a fixed 5000-password subset of RockYou, disjoint from every oracle corpus and split 80/10/10 into training, validation, and test partitions; its training set is built by running HoneyGen Baseline with $k = 20$ on the 4000 real passwords of the training partition, producing 19 honeywords per real password. The attacker CNN is therefore trained on 4000 real and 76,000 honeyword examples, and this single trained attacker is reused to evaluate every generator at every training size. Training uses batch size $\text{BS}_{\text{CNN}} = 256$ and runs for ten epochs for both oracle and attacker roles.

4.4.3 Dual Role and Seed Separation

The two networks share the architecture and training procedure just described but differ in three respects. First, they are trained on disjoint corpora (Section 4.3). Second, the attacker uses a seed offset of 7777 relative to the oracle, so the two networks are not identical even where their training procedures coincide. Third, only the oracle interacts with the filter at storage time; the attacker is invoked exclusively at evaluation time and never observes the acceptance decisions the oracle made.

4.4.4 Robustness Attackers

The CNN above is the attacker the filter is tuned against. To test whether the flatness it enforces survives a change of attacker, we evaluate the accepted sets against two further attackers that the filter never saw. Both are used only at evaluation time, on sets the filter has already accepted; neither takes part in filtering.

Frequency ranker. The frequency attacker is a classical, non-learned baseline. It first builds a frequency table from the attacker corpus by counting how often each password occurs. To rank a sweetword set it scores each of the k candidates by that count — a common password scores high, a rare or never-seen one scores low (unseen candidates receive the lowest score) — and then guesses the candidates from most to least frequent, on the assumption that a real password is more likely to be a popular one. It uses no neural network and no training beyond counting. This is exactly why it is a useful control: it is the ranker-style attacker that most earlier honeyword evaluations assumed, so comparing it against the CNN shows how much a learned discriminator adds over simple popularity.

Mixed-corpus CNN. The mixed-corpus attacker has the same architecture and training procedure as the oracle CNN, but its negative examples are drawn from several generator families together — Chaffing-by-Tweaking, the Password-Model, HoneyGen, and the GAN — instead of from HoneyGen Baseline alone. Training on a broader mix stops the attacker from over-fitting to one decoy family and lets it flag sets, such as the GAN’s, that a single-family CNN misses (Section 5.7).

4.5 Honeyword Generation Methods

We benchmark ten honeyword generators. Six of them are unfiltered baselines that expose how each generation strategy behaves against the learned attacker; four are filtered variants that combine a generator with the pre-screening or rejection sampling stages of Section 4.6.

4.5.1 Chaffing Family

The three *chaffing* methods derive candidates directly from w^* and do not perform any oracle-gated selection.

Chaffing-by-Tweaking. Rule-based edits applied to w^* : character-class substitutions on letters, digits, and symbols; suffix and prefix modifications; and short case flips. Candidates preserve the length structure of the real password.

Chaffing-with-Password-Model. Nearest neighbors of w^* in the FastText embedding space, using the skipgram model with `minCount` = 1 and `minn` = 2 trained on the oracle corpus. Candidates share sub-word structure with w^* but need not preserve its length.

Chaffing-with-Hybrid-Model. This generator combines the two streams above. For a real password w^* it draws candidates from both sources — the length-preserving rule-based edits of Chaffing-by-Tweaking and the FastText nearest neighbors of Chaffing-with-Password-Model — and forms the $k - 1$ decoys of each sweetword set from this combined pool. A resulting set can therefore contain both surface-level variants of w^* (from the tweaking stream, which keep its length and character structure) and sub-word-similar neighbors (from the password-model stream, which share its morphology but not necessarily its length). Mixing the two makes the accepted sets harder for an attacker to characterize with a single heuristic, since no one edit rule or embedding-distance cutoff explains all the decoys. Because both streams reuse the same underlying components as

HoneyGen Baseline, the random seed that governs how the two are combined is offset by 5000 from the base seed; this ensures the hybrid method produces a distinct sample rather than reproducing the sets of HoneyGen Baseline.

4.5.2 HoneyGen Family

The five *HoneyGen* methods build on the algorithm of Dionysiou, Vassiliades, and Athanasopoulos [DVA21], which combines the FastText password model with rule-based tweaking and applies frequency-based pruning to the resulting candidate pool.

HoneyGen Baseline. The HoneyGen algorithm as described by Dionysiou, Vassiliades, and Athanasopoulos [DVA21], without any oracle interaction. This is the reference point against which the effect of every filter is measured.

HoneyGen Set-Filter. HoneyGen Baseline followed by set-level rejection sampling only. Candidate honeywords are used as produced; the oracle decides acceptance at the sweetword-set level.

HoneyGen Prescreen-Add and Prescreen-Ratio. HoneyGen Baseline restricted to its password-model component (the `honeygen-model-only` configuration), followed by both candidate-level pre-screening and set-level rejection sampling. The Add variant uses the additive band condition $|s(w) - s(w^*)| \leq \tau_{\text{band}}$, and the Ratio variant uses the two-sided multiplicative band condition $\frac{1}{\tau_{\text{band-ratio}}} s(w^*) \leq s(w) \leq \tau_{\text{band-ratio}} s(w^*)$ with $\tau_{\text{band-ratio}} > 1$, both defined in Section 4.6.

HoneyGen Hybrid-Add and Hybrid-Ratio. Identical to the Prescreen variants except that the candidate stream is the full HoneyGen hybrid (password model plus tweaking) rather than password-model only. Both use the two-stage filter.

4.5.3 GAN

A generative adversarial network trained separately for each training-set size. Candidates are drawn from the trained generator, followed by oversampling by a factor of approximately five, deduplication, and removal of malformed entries; a random subset of the survivors is retained as the $k - 1$ honeywords. The GAN is not gated by the oracle. Its results in Chapter 5 require a qualification that we discuss there.

4.6 Hybrid Flatness-Filtering Algorithm

The filter has two stages. Candidate-level pre-screening removes individually distinguishable decoys before a sweetword set is assembled, and set-level rejection sampling discards sets whose induced distribution violates one of the flatness conditions from Section 4.2. Both stages use the oracle CNN of Section 4.4.

4.6.1 Candidate-Level Pre-Screening

For each candidate honeyword w , the oracle produces a score $s(w) \in [0, 1]$. The pre-screening condition compares this score to the score $s(w^*)$ of the real password within a band around it: the *additive* band accepts w when

$$|s(w) - s(w^*)| \leq \tau_{\text{band}}, \quad (4.10)$$

and the *multiplicative* band accepts w when its score lies within a factor $\tau_{\text{band-ratio}}$ of $s(w^*)$ on either side,

$$\frac{1}{\tau_{\text{band-ratio}}} s(w^*) \leq s(w) \leq \tau_{\text{band-ratio}} s(w^*), \quad \tau_{\text{band-ratio}} > 1. \quad (4.11)$$

The two variants correspond to the Prescreen and Hybrid families of Section 4.5. Both bands are two-sided: they remove candidates that the oracle is confident about in either direction — whether obviously honeyword-looking, with a score far below $s(w^*)$, or obviously real-looking, with a score far above it — since either kind stands out in the induced distribution.

It is worth being explicit about the scale of s . The oracle is a binary classifier, so $s(w)$ is its confidence that w is a real password, not a probability over the password space; a value of 0.2 means the classifier is 20% confident, which is an ordinary output, not an implausibly popular password. Because the attacker corpus contains one real password per $k = 20$ candidates, the classifier’s base rate is $1/k = 0.05$, and measured scores for real passwords cluster around 0.20–0.25 with a fifth percentile near 0.12. One practical adjustment is still needed for the multiplicative band: when the oracle is undertrained it occasionally scores a real password close to zero, which would shrink the band to almost nothing and reject every candidate. The band is therefore built around $\max(s(w^*), 0.05)$, with the floor set at the base rate. In our runs this floor binds on fewer than 1% of passwords, so for almost every set the band is centred on $s(w^*)$ itself.

4.6.2 Set-Level Rejection Sampling

After $k - 1$ candidates survive pre-screening, the sweetword set $S = \{w^*\} \cup \{w_1, \dots, w_{k-1}\}$ is passed to the oracle, which produces the score vector $s = (s_1, \dots, s_k)$ and, via Equation (4.2), the induced distribution q . The set is accepted if all three of the following hold:

$$q_{\max} \leq \frac{1}{k} + \varepsilon_{\max}, \quad (4.12)$$

$$F_{\text{ent}} \geq 1 - \delta_H, \quad (4.13)$$

$$\Delta_{\text{top}} \leq \text{gap}_{\max}. \quad (4.14)$$

If any condition fails, the entire set is discarded and a fresh one is generated. Regeneration continues until acceptance or until a regeneration cap $R_{\max}$ is reached, in which case the best set observed so far (measured by minimum $q_{\max}$) is retained.

There is one design limitation of this test worth stating explicitly. The predicate caps only the largest entry $q_{\max}$; it imposes no lower bound on any entry. A set in which some honeyword receives an anomalously low q can therefore still pass, even though the resulting distribution is not perfectly uniform. In practice this asymmetry does not affect ASR, because ASR responds to the top of the ranking rather than the tail; we return to the point in Chapter 5.

4.6.3 Algorithm

The combined procedure is stated in Algorithm 1. The pseudocode uses generic band operator $\text{band}(\cdot)$ that stands for either the additive or the ratio variant of the pre-screening rule.

Algorithm 1 Hybrid Flatness Filtering

Require: real password w^* , generator G , oracle A , thresholds τ_{band} , ε_{max} , δ_H , gap_{max} , cap R_{max}

Ensure: sweetword set S

```
1:  $r \leftarrow 0$ ;  $S^{\text{best}} \leftarrow \emptyset$ ;  $q^{\text{best}} \leftarrow +\infty$ 
2: while  $r < R_{\text{max}}$  do
3:    $C \leftarrow G.\text{GENERATE}(w^*, N_{\text{over}})$  ▷ oversampled candidate pool
4:    $C \leftarrow \{w \in C : \text{BAND}(s(w), s(w^*))\}$  ▷ candidate pre-screening
5:   if  $|C| < k - 1$  then
6:      $r \leftarrow r + 1$ ; continue
7:   end if
8:    $S \leftarrow \{w^*\} \cup \text{TAKEFIRST}(C, k - 1)$ 
9:   compute  $s_i = A(w_i)$  and  $q_i$  via Equation (4.2)
10:  compute  $q_{\text{max}}$ ,  $F_{\text{ent}}$ , and  $\Delta_{\text{top}}$ 
11:  if  $q_{\text{max}} \leq 1/k + \varepsilon_{\text{max}}$  and  $F_{\text{ent}} \geq 1 - \delta_H$  and  $\Delta_{\text{top}} \leq \text{gap}_{\text{max}}$  then
12:    return  $S$ 
13:  end if
14:  if  $q_{\text{max}} < q^{\text{best}}$  then
15:     $S^{\text{best}} \leftarrow S$ ;  $q^{\text{best}} \leftarrow q_{\text{max}}$ 
16:  end if
17:   $r \leftarrow r + 1$ 
18: end while
19: return  $S^{\text{best}}$ 
```

4.6.4 Complexity

Let C_{CNN} denote the cost of one forward pass through the oracle and let $\mathbb{E}[R]$ denote the expected number of regeneration attempts per password. The expected cost of producing one accepted sweetword set is

$$\mathcal{O}(\mathbb{E}[R] \cdot k \cdot C_{\text{CNN}}), \quad (4.15)$$

plus the cost of the generator itself. Since C_{CNN} dominates the arithmetic cost of any of the generators used here, the filter's total cost is close to $\mathbb{E}[R] \cdot k$ oracle forward passes per password. Batched scoring keeps this modest for $k = 20$ and the R_{max} values reported in Section 4.8, and filtering happens once at storage time rather than at every authentication.

4.7 Evaluation Protocol

4.7.1 Design

Each generator is trained once at a fixed training size N_{train} , and every evaluation set is produced by that single trained generator. $\text{ASR}@t$ is then computed over a test set of sweetword sets, restricted to the sets the filter accepted (Section 4.6). Because the generator is fixed for a given N_{train} , the reported figures compare methods on the same footing rather than conflating a change of generator with a change of measurement. We report results at three training sizes, $N_{\text{train}} \in \{2000, 5000, 10000\}$, evaluating each on a test set of 2000 sweetword sets.

4.7.2 Rational Attacker Floor

The CNN attacker’s raw $\text{ASR}@t$ — the success of following the CNN’s t highest-scored candidates — can fall *below* the random-guessing baseline t/k on some sets. This is not a sign that the set is safe. It means the CNN’s ranking is *anti-informative*: it places the real password near the *bottom* of the order, so following its top guesses does worse than chance. A rational attacker who observed this would not keep following the CNN; it would invert its strategy and guess the candidates the CNN scores *lowest*, which then succeeds far more often than chance.

We therefore report the success of a rational attacker that uses the CNN ranking in whichever direction helps, falling back on random guessing when neither direction is informative:

$$\text{ASR}_{\text{reported}}(t) = \max(\text{ASR}_{\text{CNN}}(t), \text{ASR}_{\overline{\text{CNN}}}(t), t/k), \quad (4.16)$$

where $\text{ASR}_{\text{CNN}}(t)$ counts the sets whose real password is among the CNN’s top t candidates, $\text{ASR}_{\overline{\text{CNN}}}(t)$ counts those whose real password is among the CNN’s *bottom* t candidates (the inverted strategy), and t/k is plain random guessing. A genuinely flat generator drives all three terms to t/k , so the reported value is t/k . A generator whose real password the CNN systematically ranks low — notably the GAN of Section 5.7 — is caught by the inverted term, which reports the high success such a set really allows instead of masking it at t/k . This is the value reported throughout Chapter 5; the GAN-focused comparisons in Section 5.7 additionally break it into its follow-the-CNN and inverted components, and report the raw per-attacker success, so that the inversion behind the GAN’s number is visible rather than hidden.

4.8 Experimental Protocol and Parameters

4.8.1 Locked Parameters

The following parameters are fixed for every experiment reported in Chapter 5. The flatness thresholds were set in two stages. An initial, strict configuration ($\varepsilon_{\max} = 0.02$, $\delta_H = 0.04$, $\text{gap}_{\max} = 0.01$) was chosen from the theoretical rationale in Section 4.2 and preliminary runs on the training split. As reported in Section 4.8.2, this configuration drove the ratio-based filters to acceptance rates below 10%, at which point the filtered methods can no longer be meaningfully compared to the unfiltered baselines. We therefore adopted a relaxed configuration, listed below, that restores usable acceptance across all filters at every corpus size. None of the thresholds was tuned on the evaluation split; the revision was driven entirely by the acceptance-rate diagnostic on the training split.

Symbol	Value	Role
k	20	Sweetword set size
$\varepsilon_{\max}$	0.05	ε -flatness bound (see note below)
δ_H	0.10	Entropy slack, $F_{\text{ent}} \geq 1 - \delta_H$
$\text{gap}_{\max}$	0.05	Top-gap threshold
τ_{band}	0.15	Additive pre-screening band
$\tau_{\text{band-ratio}}$	2.0	Multiplicative pre-screening band
$R_{\max}$	50	Regeneration cap
BS_CNN	256	CNN training batch size
Epochs (oracle)	10	CNN oracle training epochs
Epochs (attacker)	10	CNN attacker training epochs
SEED	42	Base random seed
Attacker seed offset	7777	Added to base seed for attacker
Hybrid model seed offset	5000	Added to base seed for Hybrid-Model

A single value of $\varepsilon_{\max} = 0.05$ is used throughout, both in the definition of ε -flatness (Equation (4.7)) and in the filter’s acceptance predicate (Section 4.6), which share the additive form $q_{\max} \leq 1/k + \varepsilon_{\max}$.

4.8.2 Threshold Selection and the Accept-Rate / ASR Trade-off

The flatness thresholds control a trade-off between two quantities that a deployed countermeasure must satisfy simultaneously. A stricter gate produces flatter accepted sets, lowering ASR, but rejects a larger share of passwords; each rejected password reaches the

regeneration cap without an accepted set and is excluded from the reported ASR, which is therefore conditional on acceptance. The accept rate measures how often the filter succeeds, and the ASR measures how flat the sets are when it does.

Under the initial thresholds, the ratio-based filters accepted fewer than 10% of candidate passwords at $N_{\text{train}} \in \{5000, 10000\}$ (Set-Filter reached 1.2% at $N_{\text{train}} = 5000$), which makes their ASR a statement about a small and self-selected minority of passwords rather than about the method as a deployed defense. The adopted thresholds raise every filter above 25% acceptance at every corpus size (Chapter 5), at the cost of a modest increase in ASR on the accepted sets. This is a deliberate choice: a filter that protects a single-digit percentage of users is not a usable countermeasure regardless of how flat its accepted sets are, so acceptance is treated as a first-class deployability requirement rather than a secondary diagnostic.

Consequently, methods are compared in two stages in Chapter 5. A method must first clear a usable acceptance threshold; only among the methods that do so is the flattest (lowest-ASR) one preferred. Under this criterion the ratio-based filters are the recommended configurations, since they deliver the flattest sets among the methods that remain deployable, whereas the additive filters accept more passwords but sit closer to the unfiltered baseline in flatness.

4.8.3 Metrics and Statistical Treatment

For each generator and each N we report ASR@1, ASR@3, and ASR@5 using Equation (4.16), with ASR@5 as the primary metric. Alongside each ASR value we report the mean and median of F_{rank} , F_{ent} , q_{real} , and Δ_{top} over the evaluation sets, together with the number of accepted sets for the filtered generators. Confidence intervals at small N are wide, approximately ± 20 percentage points at $N \leq 500$, and we do not draw conclusions from small- N comparisons that fall inside this margin.

4.8.4 Robustness Check

The primary corpus for all experiments is RockYou, and the primary attacker is the HoneyGen-tuned CNN. Robustness is assessed by re-evaluating the accepted sets against two further attacker classes — a mixed-corpus CNN trained on several decoy families and a frequency ranker — as reported in Section 5.8. Extending the benchmark to a second corpus such as *darkc0de* is left as future work (Chapter 6).

Chapter 5

Results

This chapter reports the experiments defined in Section 4.7. Section 5.1 recaps the setup and how to read the tables. Section 5.2 shows how vulnerable the unfiltered honeyword generators are to the CNN attacker of Section 4.4. Section 5.3 isolates the effect of the two-stage filter of Section 4.6 on ASR@ t and on the flatness criteria of Section 4.2. Section 5.4 reports the acceptance rates and regeneration counts and develops the deployability-first reading of the results. Section 5.5 confirms that the ordering of methods is preserved across training-set sizes. Section 5.7 isolates a distributional artifact of the GAN. Section 5.8 evaluates the accepted sets against a frequency attacker as a cross-attacker robustness check. Section 5.9 draws the picture together.

5.1 Setup

All experiments are run on the cleaned RockYou corpus. For each training size $N_{\text{train}} \in \{2000, 5000, 10000\}$ a separate oracle and generator are trained, and every trained generator produces sweetword sets that are evaluated on a test set of 2000 sweetword sets (Section 4.7). The attacker CNN is trained once on a disjoint 5000-password attacker corpus with HoneyGen-Baseline negatives and reused unchanged across all training sizes and generators (Section 4.4), so that a single, consistent adversary evaluates every method. All parameter values are those of Table 5.1 in Section 4.8, i.e. the relaxed thresholds adopted in Section 4.8.2.

The primary security metric is ASR@5: the fraction of sets on which the attacker’s five highest-ranked guesses include the real password. Under ideal flatness $\text{ASR@5} = 5/k = 0.25$ for $k = 20$; higher values quantify how much better than random guessing the attacker does. We also report the flatness criteria F_{rank} (the normalised rank of the real password;

Symbol	Meaning	Value
k	Sweetword set size	20
$\varepsilon_{\max}$	ε -flatness bound ($q_{\max} \leq 1/k + \varepsilon_{\max}$)	0.05
δ_H	Entropy slack ($F_{\text{ent}} \geq 1 - \delta_H$)	0.10
$\text{gap}_{\max}$	Top-gap threshold	0.05
τ_{band}	Additive pre-screening band	0.15
$\tau_{\text{band-ratio}}$	Multiplicative pre-screening band	2.0
$R_{\max}$	Regeneration cap	50
BS_CNN	CNN training batch size	256
Epochs (oracle / attacker)	CNN training epochs	10 / 10
Epochs (GAN final / quick)	GAN training epochs	60 / 20
Epochs (FastText final / quick)	FastText training epochs	100 / 20

Table 5.1: Adopted parameter values for all experiments in Chapter 5. The flatness thresholds are the relaxed configuration of Section 4.8.2.

0.5 is the ideal middle), F_{ent} (normalised entropy of the induced distribution; close to 1 is uniform), and F_{ε} (the ε -flatness pass rate). Table 5.1 lists every fixed parameter. Unless stated otherwise, the detailed single-training-size tables use $N_{\text{train}} = 10000$, the largest and most stable corpus; Section 5.5 shows the smaller sizes reproduce the same ordering.

5.2 Unfiltered Baselines Under the CNN Attacker

The four unfiltered honeyword-generation strategies (Chaffing-by-Tweaking, Chaffing-with-Password-Model, Chaffing-with-Hybrid-Model, HoneyGen Baseline) are all highly distinguishable to the CNN attacker. At $N_{\text{train}} = 10000$ they report ASR@5 of 0.678, 0.755, 0.743, and 0.746 respectively (Table 5.2), roughly three times the random-guessing rate of 0.25. Two points follow. First, the four strategies cluster tightly: the three model-based generators are within 0.02 of one another, and the simplest rule-based generator, Chaffing-by-Tweaking, is only about 0.07 lower. Second, the more elaborate statistical machinery of the hybrid and HoneyGen generators does *not* translate into flatness against a learned discriminator: their F_{rank} values sit near 0.169, meaning the real password is ranked near the top of the twenty candidates on the average set. This is the pattern that motivates the filter.

Method	$N_{\text{train}}=2000$			$N_{\text{train}}=5000$			$N_{\text{train}}=10000$		
	@1	@3	@5	@1	@3	@5	@1	@3	@5
Chaffing-by-Tweaking	0.290	0.544	0.679	0.287	0.539	0.678	0.281	0.540	0.678
Chaffing-with-Password-Model	0.303	0.615	0.759	0.300	0.617	0.767	0.304	0.600	0.755
Chaffing-with-Hybrid-Model	0.341	0.611	0.756	0.338	0.620	0.752	0.337	0.603	0.743
HoneyGen Baseline	0.339	0.614	0.752	0.336	0.619	0.761	0.346	0.604	0.746
HoneyGen + Set-Filter	0.305	0.583	0.728	0.115	0.323	0.518	0.179	0.438	0.610
HoneyGen + Prescreen-Add	0.164	0.412	0.588	0.109	0.299	0.470	0.144	0.368	0.525
HoneyGen + Prescreen-Ratio	0.083	0.262	0.417	0.071	0.205	0.341	0.071	0.217	0.341
HoneyGen + Hybrid-Add	0.222	0.473	0.641	0.093	0.299	0.462	0.141	0.364	0.510
HoneyGen + Hybrid-Ratio	0.139	0.350	0.503	0.083	0.172	0.285	0.050	0.167	0.293
GAN (Self-Trained)	0.101	0.222	0.331	0.565	0.718	0.774	0.745	0.832	0.872
Random guessing (t/k)	0.050	0.150	0.250	0.050	0.150	0.250	0.050	0.150	0.250

Table 5.2: ASR@ t under the CNN attacker for all ten methods at three training sizes, reported as the rational-attacker value of Equation (4.16); no entry falls below the random-guessing baseline t/k . Columns are ASR@1, ASR@3, ASR@5. Each block is evaluated on a test set of sweetword sets produced by the generator trained at that N_{train} .

5.3 Effect of Flatness Filtering

Table 5.2 reports ASR@1, ASR@3, and ASR@5 for all ten methods at the three training sizes, and Table 5.3 consolidates the $N_{\text{train}} = 10000$ column with the flatness criteria and the filtering cost. Reading the ASR@5 column at $N_{\text{train}} = 10000$, the methods separate into three families.

Unfiltered baselines. The four baselines occupy the vulnerable band, ASR@5 between 0.678 and 0.755, and are the reference against which the filters are measured.

Additive filters. Set-Filter, Prescreen-Add, and Hybrid-Add apply the set-level acceptance test of Section 4.6 either alone (Set-Filter) or with a symmetric additive band on the candidate scores. They cut ASR@5 to between 0.510 and 0.610, a substantial reduction that nonetheless leaves the attacker well above the random-guessing rate: on roughly half of the accepted sets the CNN’s top five guesses still contain the real password. Their F_{rank} rises to about 0.296, part-way toward the ideal 0.5.

Method	ASR@5	F_{rank}	F_{ent}	F_{ϵ}	Accept	Tries
Chaffing-by-Tweaking	0.678	0.208	0.984	0.207	100%	1.00
Chaffing-with-Password-Model	0.755	0.171	0.971	0.125	100%	1.00
Chaffing-with-Hybrid-Model	0.743	0.171	0.970	0.132	100%	1.00
HoneyGen Baseline	0.746	0.169	0.970	0.130	100%	1.00
HoneyGen + Set-Filter	0.610	0.235	0.979	0.189	54.5%	3.51
HoneyGen + Prescreen-Add	0.525	0.296	0.990	0.329	84.6%	1.00
HoneyGen + Prescreen-Ratio	0.341	0.422	0.992	0.505	54.1%	1.00
HoneyGen + Hybrid-Add	0.510	0.296	0.989	0.328	83.3%	1.26
HoneyGen + Hybrid-Ratio	0.293	0.462	0.991	0.569	47.9%	2.68
GAN (Self-Trained)	0.872	0.920	0.989	0.958	100%	1.00

Table 5.3: Consolidated view at $N_{\text{train}} = 10000$ under the CNN attacker: security (ASR@5, lower is flatter), flatness criteria (F_{rank} and F_{ϵ} near 0.5 and high F_{ent} indicate near-uniform sets), and cost (accept rate and average regeneration attempts). Unfiltered methods have no filter and are shown at 100% / 1.00.

Ratio filters. Prescreen-Ratio and Hybrid-Ratio use the two-sided multiplicative band $\frac{1}{\tau_{\text{band-ratio}}} s(w^*) \leq s(w) \leq \tau_{\text{band-ratio}} s(w^*)$ during candidate screening. They are the strongest genuine defence, pushing ASR@5 down to 0.341 (Prescreen-Ratio) and 0.293 (Hybrid-Ratio) and lifting F_{rank} to 0.422 and 0.462 respectively — close to the ideal middle value of 0.5 that characterises a near-uniform ranking. Unlike the GAN (Section 5.7), the ratio filters reach low ASR *and* a central F_{rank} at the same time, which is what distinguishes genuine flatness from inverted distinguishability.

Table 5.3 makes the ordering explicit and adds the cost of each filter. The relative ordering of the six filtered methods is stable across ASR@1, ASR@3, and ASR@5: ratio variants beat their additive counterparts, and Set-Filter — which uses only set-level rejection without candidate-level pre-screening — is the weakest filtered method, ...confirming that the candidate-level pre-screening stage contributes rejection capability the set-level test alone does not recover. The table also lists the GAN for completeness; its high ASR@5 together with an F_{rank} near 1 is the inverted-distinguishability artifact analysed separately in Section 5.7, and it should not be read as a filtered defence.

5.4 Cost of Filtering and the Deployability Gate

The security ranking in Table 5.3 is only half the picture: a filter is useful only if it accepts a workable fraction of passwords. Table 5.4 reports the acceptance rate and the average number of regeneration attempts per accepted set, for the five filtered methods at each

Method	$N_{\text{train}}=2000$		$N_{\text{train}}=5000$		$N_{\text{train}}=10000$	
	Accept	Tries	Accept	Tries	Accept	Tries
HoneyGen + Set-Filter	90.6%	1.56	25.9%	4.62	54.5%	3.51
HoneyGen + Prescreen-Add	94.8%	1.00	68.4%	1.00	84.6%	1.00
HoneyGen + Prescreen-Ratio	65.5%	1.00	38.7%	1.00	54.1%	1.00
HoneyGen + Hybrid-Add	96.5%	1.03	65.5%	1.49	83.3%	1.26
HoneyGen + Hybrid-Ratio	77.5%	1.70	29.3%	3.55	47.9%	2.68

Table 5.4: Acceptance rate and average regeneration attempts per accepted set for the five filtered HoneyGen variants under the adopted thresholds. Unfiltered methods have no filter (100% acceptance, one attempt) and are omitted. Every filtered method now exceeds 25% acceptance at every training size.

training size, under the adopted thresholds.

Two readings follow. First, there is a clear efficiency–security trade-off: the additive filters accept the large majority of candidate sets (between 65.5% and 96.5% at $N_{\text{train}} = 2000$) in close to a single attempt, whereas the ratio filters — the strongest defence — accept a smaller fraction and cost more regeneration attempts. This is the trade-off introduced in Section 4.8.2: a stricter effective gate buys flatness at the price of acceptance.

Second, and decisively for the thesis’s recommendation, the adopted thresholds keep every filtered method above the 25% acceptance mark at every training size. Under the initial strict thresholds the ratio filters fell below 10% acceptance — Set-Filter reached 1.2% at $N_{\text{train}} = 5000$ — at which point their reported ASR described only a small, self-selected minority of passwords and could not be compared with the unfiltered methods. We therefore read the results in two stages, following Section 4.8.2. A method must first clear a usable acceptance rate to count as deployable; only among the deployable methods is the flattest (lowest-ASR) one preferred. Under this criterion the ratio filters are the recommended configurations: they deliver the flattest genuinely-flat sets while remaining deployable at every corpus size, whereas the additive filters accept more passwords but sit closer to the unfiltered baseline in flatness.

Filtering happens once per password at storage time, not on every authentication (Section 4.1). Even the highest average regeneration counts in Table 5.4, for $k = 20$, amount to a sub-second cost on a small GPU and are amortised over the lifetime of the credential. A deployment that cannot afford the ratio filter’s rejection budget can fall back to an additive filter and still recover most of the security gain over the unfiltered baseline.

5.5 Stability Across Training-Set Size

Table 5.2 reports every method at all three training sizes. The three-family structure is preserved throughout. The unfiltered baselines change ASR@5 by at most about 0.02 across the three corpus sizes: the CNN’s ability to distinguish real passwords from HoneyGen-family decoys does not depend meaningfully on how much data the generator saw. The additive and ratio filters likewise keep their relative order at every size, with the ratio filters remaining the flattest genuine defence throughout — Prescreen-Ratio moves over the range 0.417–0.341 and Hybrid-Ratio over 0.503–0.293 across the three sizes. The mild upward drift of the ratio filters at intermediate sizes reflects a shrinking oracle/attacker agreement as the two networks are trained on more data, not a failure of the filter; Chapter 6 returns to this boundary of model-relative flatness.

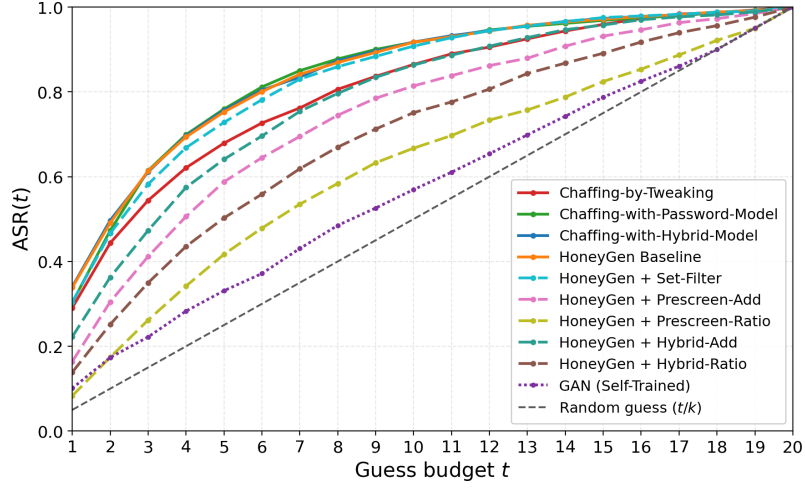
5.6 ASR Curves and Consistency Across Test-Set Size

Figure 5.1 shows the ASR curves under the CNN attacker for all ten methods at the three training sizes, each evaluated on the same test set of 2000 sweetword sets. The curves make the three-family structure visible: the unfiltered baselines sit high, the additively filtered methods fall to an intermediate band, and the ratio filters lie closest to the random-guessing diagonal. The GAN is the outlier at the top: once the rational-attacker rule of Equation (4.16) is applied, its anti-informative ranking is exploited rather than rewarded, and it becomes the most attackable method in the benchmark (Section 5.7).

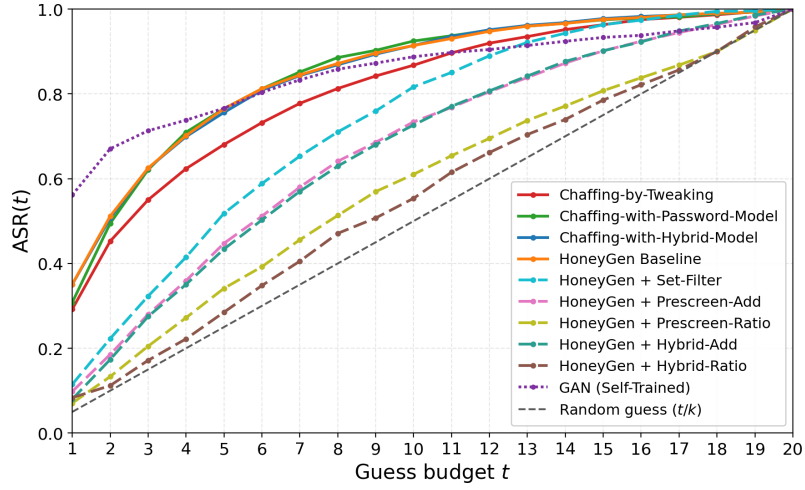
5.7 The GAN Result Requires a Caveat

The GAN’s reported $\text{ASR@5} = 0.872$ at $N_{\text{train}} = 10000$ (Table 5.2) makes it one of the weakest methods in the benchmark, but the reason is worth spelling out, because an attacker that simply followed the CNN would conclude the opposite. Table 5.5 reports two attacker strategies side by side together with F_{rank} .

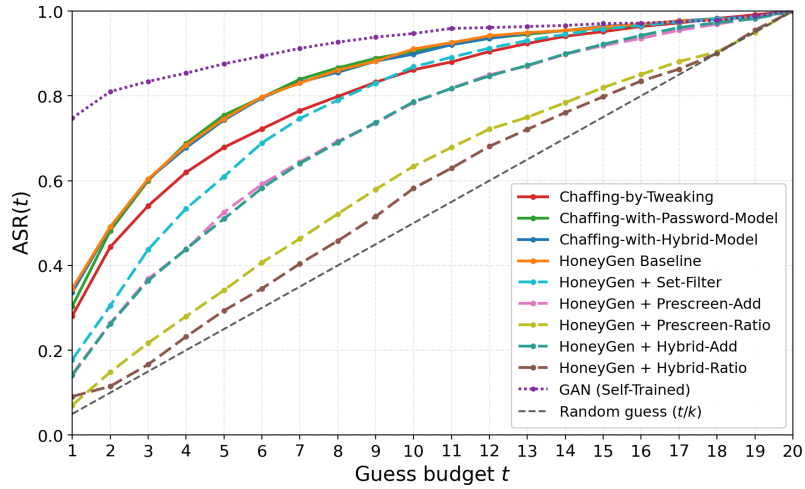
For HoneyGen Baseline, following the CNN’s top-ranked guesses succeeds 74.6% of the time and F_{rank} is 0.169 — the real password sits near the top, so the attacker just follows the CNN. For the ratio filters both attacker directions land near the random-guessing rate and F_{rank} is close to the ideal 0.5; the real password is in the middle, so neither direction helps and the sets are genuinely flat. The GAN is the exception. Following the CNN succeeds only 2.5% of the time, which taken alone would make the GAN look like the flattest method in the benchmark, but its F_{rank} is 0.920: the real password is ranked at or near the bottom of almost every GAN set. A rational attacker inverts and guesses the CNN’s *lowest*-scored



(a) $N_{\text{train}} = 2000$.



(b) $N_{\text{train}} = 5000$.



(c) $N_{\text{train}} = 10000$.

Figure 5.1: ASR curves under the CNN attacker for all ten methods, each evaluated on the same test set of 2000 sweetword sets, at the three training sizes. Curves report the rational-attacker value of Equation (4.16), so no method falls below the random-guessing diagonal. Solid lines are unfiltered generators, dashed lines are filtered variants, and the dotted line is the GAN.

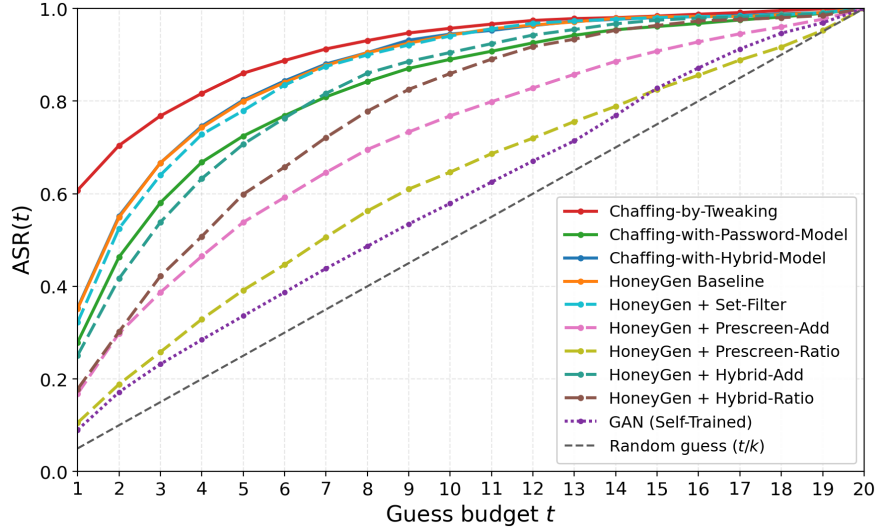


Figure 5.2: ASR curves under the mixed-corpus attacker (trained on Tweak + Password-Model + HoneyGen + GAN negatives) at $N_{\text{train}} = 2000$, test on 2000 sweetword sets. Colours and line styles match Figure 5.1, and curves report the rational-attacker value of Equation (4.16).

Method	Follow-CNN ASR@5	Anti-CNN ASR@5	Reported ASR@5	F_{rank}
HoneyGen Baseline	0.746	0.038	0.746	0.169
HoneyGen + Prescreen-Ratio	0.341	0.180	0.341	0.422
HoneyGen + Hybrid-Ratio	0.293	0.202	0.293	0.462
GAN (Self-Trained)	0.025	0.872	0.872	0.920

Table 5.5: Two attacker strategies at $N_{\text{train}} = 10000$: *Follow-CNN* guesses the CNN’s top-ranked candidates, *Anti-CNN* guesses its bottom-ranked ones, and *Reported* is the rational-attacker value of Equation (4.16). For the baseline and the ratio filters, Follow-CNN is the larger of the two directions and F_{rank} sits near or above 0.4, so the reported value simply follows the CNN. The GAN is the exception — its F_{rank} near 1 places the real password at the bottom, so Follow-CNN succeeds only 2.5% of the time while Anti-CNN reaches 87%. Its low Follow-CNN score is inverted distinguishability, not flatness.

candidates, and succeeds 87.2% of the time — which is the value the two-sided floor of Equation (4.16) reports. A GAN trained on the oracle corpus samples from the corpus’s high-probability region, and the real password, itself a low-probability sample, looks like an outlier the classifier confidently rejects; this is inverted distinguishability, not flatness.

Method	CNN-only ASR@5	Mixed-corpus ASR@5
Chaffing-by-Tweaking	0.679	0.860
Chaffing-with-Password-Model	0.759	0.725
Chaffing-with-Hybrid-Model	0.756	0.802
HoneyGen Baseline	0.752	0.799
HoneyGen + Set-Filter	0.728	0.779
HoneyGen + Prescreen-Add	0.588	0.538
HoneyGen + Prescreen-Ratio	0.417	0.392
HoneyGen + Hybrid-Add	0.641	0.707
HoneyGen + Hybrid-Ratio	0.503	0.599
GAN (Self-Trained)	0.212	0.336

Table 5.6: ASR@5 under the HoneyGen-only CNN attacker versus the mixed-corpus attacker (Tweak + Password-Model + HoneyGen + GAN negatives) at $N_{\text{train}} = 2000$. The GAN moves off the near-floor value it reported under the CNN-only attacker ($0.21 \rightarrow 0.34$), while the filtered ratio methods shift comparatively little; the effect is much larger at $N_{\text{train}} = 5000$ (Section 5.7).

The bias is confirmed by a change of attacker. Under the mixed-corpus attacker of Section 4.4 — trained on Tweak, Password-Model, HoneyGen and GAN negatives together — the GAN leaves the random-guessing diagonal it hugged under the single-family CNN attacker: at $N_{\text{train}} = 2000$ its ASR@5 rises from 0.21 to 0.34 (Figure 5.2), and the effect grows with training size, reaching 0.99 at $N_{\text{train}} = 5000$, while the filtered ratio methods shift comparatively little. Table 5.6 lists the per-method ASR@5 under the two attackers at $N_{\text{train}} = 2000$. Whether it is exposed by the inverted CNN strategy or by a broader attacker, the GAN’s apparent flatness under a naive single-family attacker is not a property of the sweetword sets it produces.

5.8 Robustness Across Attacker Class

The filter is tuned against the CNN attacker, so we check the accepted sets against the two robustness attackers of Section 4.4.4: an independent frequency ranker and a mixed-corpus CNN. Table 5.7 reports the CNN and the frequency attacker at $N_{\text{train}} = 10000$.

Three points follow. First, the frequency attacker is a substantially weaker distinguisher than the CNN: its ASR@5 on HoneyGen Baseline is 0.319 versus 0.746 for the CNN. This is the gap PassFilter identified — methods evaluated only against ranker-style attackers report flatness numbers that overstate their security once a learned discriminator is admitted. Second, filtering against the CNN does not open the sets up to the frequency

Method	CNN attacker		Freq. attacker	
	ASR@1	ASR@5	ASR@1	ASR@5
Chaffing-by-Tweaking	0.281	0.678	0.216	0.386
Chaffing-with-Password-Model	0.304	0.755	0.050	0.250
Chaffing-with-Hybrid-Model	0.337	0.743	0.083	0.319
HoneyGen Baseline	0.346	0.746	0.084	0.319
HoneyGen + Set-Filter	0.179	0.610	0.086	0.322
HoneyGen + Prescreen-Add	0.144	0.525	0.050	0.250
HoneyGen + Prescreen-Ratio	0.071	0.341	0.050	0.250
HoneyGen + Hybrid-Add	0.141	0.510	0.068	0.286
HoneyGen + Hybrid-Ratio	0.050	0.293	0.076	0.305
GAN (Self-Trained)	0.745	0.872	0.225	0.388

Table 5.7: Cross-attacker comparison at $N_{\text{train}} = 10000$ (rational-attacker ASR, Equation (4.16)). The frequency attacker separates the methods far less sharply than the CNN; the filtered HoneyGen variants stay near the frequency floor. The GAN is distinguishable under both attackers — most strongly by the CNN, which reaches 0.872 once its ranking is inverted (the real password sits at the bottom), and also by the frequency attacker at 0.388; it is flat under neither, as discussed in Section 5.7.

attacker: the filtered variants stay near the frequency floor, so there is no security trade-off between the two attacker classes. This absence of a trade-off is somewhat surprising, since tuning a filter to one attacker does not in general protect against a different one, and we do not have a proven explanation for it. A plausible one is that the features the CNN relies on to judge how “real” a candidate looks are themselves correlated with popularity — the common structural patterns that also drive the frequency ranker — so a set the CNN cannot separate tends to be one the frequency attacker cannot separate either. We report this as an empirical observation rather than a guarantee; whether the two attacker classes genuinely align, or whether a trade-off appears against a stronger ranker, is left as future work. Third, the GAN is distinguishable to both attackers rather than flat: its frequency ASR@5 of 0.388 is the highest frequency value in the table, and under the CNN a rational attacker that inverts the ranking reaches 0.872 (Section 5.7). The GAN is thus caught from both directions — by the frequency ranker directly and by the CNN once its anti-informative ranking is inverted.

5.9 Discussion

The picture is consistent across training size and attacker class, with four conclusions.

First, unfiltered honeyword generators are vulnerable to a CNN attacker. The four baselines report ASR@5 around 0.68–0.76 across all three corpus sizes, about three times the random-guessing rate, and the sophistication of the generator does not translate into flatness.

Second, filtering restores flatness, most strongly in the ratio variant. The ratio filters bring ASR@5 down to 0.341–0.293 at $N_{\text{train}} = 10000$ with F_{rank} near the ideal middle, and preserve most of that reduction at smaller sizes; the additive filters achieve a moderate reduction at a much lower rejection cost.

Third, acceptance is a first-class requirement. Under the relaxed thresholds every filtered method is deployable ($> 25\%$ acceptance) at every corpus size, and the recommended configurations are the ratio filters read through the two-stage criterion of Section 5.4: the flattest methods that remain deployable.

Fourth, the GAN’s apparent success is a distributional artifact. Following the CNN naively, the GAN looks like the flattest method, but F_{rank} far from the ideal middle value of 0.5 — 0.920, not 0 — shows the real password is ranked at the bottom rather than the top, so a rational attacker who inverts the ranking reaches $\text{ASR@5} = 0.872$ (Equation (4.16)); the same bias is exposed by the mixed-corpus attacker, and the GAN is also the most distinguishable method under the frequency attacker. Any security claim that treats the GAN as a strong flatness-preserving generator has to face this data. Chapter 6 takes these conclusions as its starting point.

Chapter 6

Conclusion

This thesis started from a mismatch. Honeyword-based breach detection depends on flatness — the promise that an attacker who has read the password file cannot identify the real credential in a sweetword set with probability better than $1/k$ — and every widely-deployed honeyword generator has been evaluated against attackers that no longer represent the state of the art. Dani et al. [Dan+24] showed that a small character-level CNN, trained as a binary classifier on real-vs-decoy pairs, breaks flatness on essentially every honeyword scheme they tested. The question this left open was whether flatness can be restored against such an attacker without redesigning the generator.

The answer we develop is affirmative but qualified. The same CNN discriminator that PassFilter used as an attack, reused unmodified as a defensive oracle at storage time, gates sweetword sets against a small set of measurable flatness conditions. Sets that fail are regenerated. Sets that pass are stored. The generator itself is not modified, and neither is the attacker; only the pipeline in which they are combined changes. On the ten-method benchmark of Chapter 5, the two-stage filter with a multiplicative pre-screening band reduces the attacker’s ASR@5 from around 0.75 for the unfiltered baselines to 0.29–0.34 for the strongest configurations at $N_{\text{train}} = 10000$, with a real password ranked near the middle of the twenty candidates ($F_{\text{rank}} \approx 0.42\text{--}0.46$). Under the thresholds adopted in Section 4.8.2, these ratio filters accept between 29% and 78% of passwords across corpus sizes — well inside the deployable range, and far above the single-digit acceptance of the initial strict configuration.

6.1 Summary of Contributions

The concrete contributions of the thesis map to Chapter 4 and Chapter 5. First, an attacker-relative definition of flatness formalized at the level of the attacker-induced distribution over a sweetword set, characterized by four measurable quantities (F_{rank} , F_{ent} , top gap, and an ε -flatness bound $q_{\text{max}} \leq 1/k + \varepsilon_{\text{max}}$ on the largest entry of the induced distribution). Second, a hybrid flatness-filtering algorithm that combines candidate-level pre-screening with set-level rejection sampling, generator-agnostic and requiring no retraining of either the underlying generator or the attacker.

The benchmark itself is the third contribution: ten honeyword generation methods spanning rule-based tweaking, representation-learning HoneyGen variants, and a self-trained GAN, evaluated under a common experimental protocol at three training sizes, and against three attacker classes — the HoneyGen-tuned CNN, a mixed-corpus CNN trained on several decoy families, and a frequency ranker — which together separate genuine flatness from single-attacker artifacts.

6.2 What the Numbers Actually Say

Three quantitative findings deserve to be extracted from Chapter 5 and stated cleanly enough for a reader in a hurry.

Unfiltered honeyword generators are highly distinguishable to the CNN. All four baselines (Tweaking, Password-Model, Hybrid-Model, HoneyGen Baseline) report ASR@5 in the range 0.68–0.77 across every training-set size we tested, about three times the random-guessing rate 0.25.

Flatness filtering reduces ASR@5 substantially, and the ratio band is what does most of the work. At $N_{\text{train}} = 10000$ the Prescreen-Ratio and Hybrid-Ratio variants push ASR@5 down to 0.341 and 0.293 with F_{rank} near the ideal middle value of 0.5 — genuine near-uniform ranking — while the additive filters achieve moderate reductions to the 0.51–0.61 range. The relative ordering of methods is preserved as the training corpus varies over 2000, 5000, and 10000. Because acceptance is treated as a first-class deployability requirement (Section 5.4), the recommended configurations are the ratio filters read through a two-stage criterion: the flattest methods among those that remain deployable.

The GAN’s apparent flatness is a distributional artifact and should not be read as a defence. Following the CNN’s top guesses, its ASR@5 at $N_{\text{train}} = 10000$ is essentially zero, but F_{rank} is 0.920: the real password is ranked near position twenty on the average set. The CNN considers the GAN-generated candidates to look *more* like real passwords than the actual real password does, so a rational attacker inverts the ranking and reaches

ASR@5 = 0.872 — the value the two-sided rational-attacker floor reports, which makes the GAN one of the weakest methods in the benchmark rather than the strongest. Two independent checks confirm the bias rather than flatness: under the frequency attacker the same GAN sets report ASR@5 = 0.388, the highest in that column, and under the mixed-corpus attacker the GAN’s ASR@5 rises from about 0.07 to 0.99 at $N_{\text{train}} = 5000$. This artifact is pronounced at the larger corpus sizes; at $N_{\text{train}} = 2000$ the GAN’s F_{rank} is closer to the middle, so the inversion is a property that sharpens as the generator is trained on more data.

6.3 Limitations

The framework has five limitations that a deployment would need to reason about explicitly.

Coverage below 100%. The most important limitation for real-world use is that the countermeasure does not protect every user. The filter stores a sweetword set only when it can certify that set as flat; when it cannot do so within the regeneration budget, it gives up on that password, and that account receives no flatness-guaranteed set. At the operating point of this thesis the strongest (ratio) filters certify only between 29% and 78% of passwords (Table 5.4), which leaves a large share of users — in the worst case the majority — without the guarantee. A deployable system, however, has to protect 100% of accounts: it cannot simply refuse to store honeywords for the users it finds difficult. As it stands the method would therefore need to be paired with a fallback for the rejected passwords (for instance an unfiltered set), which weakens the guarantee for exactly those users. Reaching near-total coverage without sacrificing flatness is the single biggest obstacle to real-world adoption, and we return to it as an open question below.

Model-relative guarantees. The filter enforces flatness against one specific attacker: the character-level CNN of Section 4.4. A different attacker — a larger CNN, a transformer, a discriminator that uses side information beyond the password string — may recover distinguishability on sets the filter has certified. The guarantees are best read as flatness against this class of learned attacker, not as flatness in general. Section 5.8 shows that the guarantees transfer partially to the frequency attacker, which is weaker; whether they transfer to stronger models is an empirical question that requires running each candidate attacker through the same benchmark.

One-sided set-level acceptance. The set-level acceptance test constrains only the maximum of the induced distribution q . A set in which some honeyword receives an

anomalously low q would pass the test even though q is not close to uniform. In our data this does not translate into ASR degradation, because ASR responds to the top of the ranking rather than the tail, but the acceptance predicate could be tightened to bound both ends of q . This is a small extension of Algorithm 1 and would be worth doing in a production build of the filter.

Acceptance-rate cost of the ratio filter. The strongest security in the benchmark, from the Prescreen-Ratio and Hybrid-Ratio filters, comes at the lowest acceptance rates among the filtered methods — between 29% and 78% across corpus sizes, against 65%–97% for the additive filters — and a higher average number of regeneration attempts, up to about 4.6 per accepted set (Table 5.4). Under the relaxed thresholds this is no longer the crippling single-digit acceptance of the initial configuration: every filtered method clears the 25% deployability mark at every training size. The residual cost is nonetheless real and falls entirely at storage time, since filtering happens once per account for the lifetime of the credential. A deployment that cannot afford the ratio filter’s rejection budget can move to an additive filter and recover most of the security gain over the unfiltered baseline at a substantially higher acceptance rate.

Corpus and language. All experiments use the cleaned RockYou corpus, a leak of primarily English-language, user-chosen web passwords from a specific consumer service. The distributional properties of this corpus — long low-probability tails, common suffixes, digit sequences at the end — are what the FastText embeddings, the GAN, and the CNN all learn to model. Whether the same numbers hold on corpora with different structural properties (enterprise credentials, non-English leaks, machine-generated passwords) is an empirical question. Extending the benchmark to *darkc0de* as a second corpus is on the immediate next-steps list for this pipeline.

6.4 Open Questions

Beyond the immediate limitations, five research questions remain open.

The first is adversarial adaptation. The filter, as constructed, is a static defense: the oracle CNN is trained once and reused. An attacker who knew the filter existed and had access to a sample of accepted sets could train against those sets rather than against the raw generator output. Whether the generator, the oracle, and the attacker can be co-trained toward a stable equilibrium at $\text{ASR}@5 \approx 0.25$ is a question this thesis does not resolve. The pieces are in place to run the experiment; the answer requires it.

The second is transferability across a class of attackers. The current guarantees are against

one CNN. A more useful guarantee would be against a class of models — for example, any CNN of bounded depth and width, or any linear classifier over a fixed feature set. The filter could then use an ensemble oracle that discriminates against the worst-case member of the class. Chakraborty et al. [Cha+22] discuss related notions in the survey, and connecting them to the flatness formalism of Section 4.2 is a natural extension.

The third is the theoretical relationship between the flatness metrics. F_{rank} , F_{ent} , and ε -flatness are correlated but not equivalent, and the acceptance predicate uses all three plus the top gap. It is not obvious that this is the smallest set of conditions sufficient to bound $\text{ASR}@t$, or that all three are individually necessary. A cleaner theoretical characterization — an inequality connecting q_{real} , F_{ent} , and expected $\text{ASR}@t$ — would allow the filter to constrain fewer quantities without giving up guarantees.

The fourth is the LLM-based generator line. The construction of Section 4.6 is generator-agnostic and would apply without modification to sweetword sets produced by the chunk-manipulation and Metropolis-Hastings large language model of the kind reviewed in Section 3.3. Whether the resulting $\text{ASR}@5$ curves match, exceed, or fall below the ratio-filtered HoneyGen numbers reported here is an experiment worth running.

The fifth is coverage. As the limitations above stress, the filter protects only the passwords for which it can certify a flat set, and at the current operating point that fraction is well below 100%. A practical countermeasure must cover every account, so the open question is how to reach near-total coverage without weakening flatness. Possible routes include stronger or password-specific generators that produce flat candidates more reliably, thresholds adapted per password to the hardest cases, or a principled fallback for the residual passwords that no generator can flatten. Which of these — or which combination — closes the gap is the most important question this thesis leaves open.

6.5 Closing

Honeywords are a modest defense with an outsized property: they turn a silent failure — offline password recovery from a stolen hash file — into an observable event at the authentication server. That property is worth preserving as attacker capabilities improve, and the argument this thesis makes is that each new class of attacker should be reused as an enforcement tool at storage time. The classifier that would otherwise recover the real password becomes the gatekeeper that prevents any sweetword set with a recoverable real password from being stored. The reversal is small and the numbers are moderate, but the direction feels right: as long as attackers keep getting better, defenders can keep the same tools, pointed the other way.

Bibliography

- [Boj+17] Piotr Bojanowski, Edouard Grave, Armand Joulin, and Tomas Mikolov. “Enriching Word Vectors with Subword Information”. In: *Transactions of the Association for Computational Linguistics (TACL)* 5 (2017), pp. 135–146.
- [Cha+22] Nilesch Chakraborty et al. “A Survey of Honeyword-Based Authentication Techniques”. In: *ACM Computing Surveys* (2022).
- [Dan+24] Dani et al. “PassFilter: Detecting Honeywords with Learned Discriminators”. In: *arXiv preprint* (2024).
- [DVA21] Antreas Dionysiou, Vassilis Vassiliades, and Elias Athanasopoulos. “HoneyGen: Generating Honeywords Using Representation Learning”. In: *Proceedings of the ACM Asia Conference on Computer and Communications Security (ASIA CCS)*. 2021, pp. 265–279.
- [FYM20] Muhammad Ali Fauzi, Bian Yang, and Edlira Martiri. “PassGAN for Honeywords: Evaluating the Defender and the Attacker Strategies”. In: *Advances on Smart and Soft Computing (ICACIn 2020)*. Vol. 1188. Advances in Intelligent Systems and Computing. Springer, 2020, pp. 391–401. DOI: [10.1007/978-981-15-6048-4_34](https://doi.org/10.1007/978-981-15-6048-4_34).
- [Goo+14] Ian J. Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. “Generative Adversarial Nets”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2014.
- [Hit+19] Briland Hitaj, Paolo Gasti, Giuseppe Ateniese, and Fernando Perez-Cruz. “PassGAN: A Deep Learning Approach for Password Guessing”. In: *Applied Cryptography and Network Security (ACNS)*. 2019, pp. 217–237.
- [JR13] Ari Juels and Ronald L. Rivest. “Honeywords: Making Password-Cracking Detectable”. In: *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 2013, pp. 145–160.
- [Kim14] Yoon Kim. “Convolutional Neural Networks for Sentence Classification”. In: *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2014, pp. 1746–1751.

- [Mel+16] William Melicher, Blase Ur, Sean M. Segreti, Saranga Komanduri, Lujo Bauer, Nicolas Christin, and Lorrie Faith Cranor. “Fast, Lean, and Accurate: Modeling Password Guessability Using Neural Networks”. In: *USENIX Security Symposium*. 2016, pp. 175–191.
- [Mik+13] Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg Corrado, and Jeffrey Dean. “Distributed Representations of Words and Phrases and their Compositionality”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2013.
- [NS05] Arvind Narayanan and Vitaly Shmatikov. “Fast Dictionary Attacks on Passwords Using Time-Space Tradeoff”. In: *Proceedings of the ACM Conference on Computer and Communications Security (CCS)*. 2005, pp. 364–372.
- [Wei+09] Matt Weir, Sudhir Aggarwal, Breno de Medeiros, and Bill Glodek. “Password Cracking Using Probabilistic Context-Free Grammars”. In: *IEEE Symposium on Security and Privacy (S&P)*. 2009, pp. 391–405.
- [YM23] Fangyi Yu and Miguel Vargas Martin. “Honey, I Chunked the Passwords: Generating Semantic Honeywords Resistant to Targeted Attacks Using Pre-trained Language Models”. In: *Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA)*. 2023.
- [ZZL15] Xiang Zhang, Junbo Zhao, and Yann LeCun. “Character-Level Convolutional Networks for Text Classification”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2015.